BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC FPT — FPTU QUY NHON AI CAMPUS

FU CAHATA · KHOA CÔNG NGHỆ THÔNG TIN — BỘ MÔN KỸ THUẬT PHẦN MỀM

OMNICAST | NỀN TẢNG PHÁT SÓNG THÔNG MINH

BÁO CÁO NGHIỆM THU ĐỒ ÁN HỌC PHẦN

HỌC PHẦN: PRN232 — XÂY DỰNG ỨNG DỤNG PHÍA SAU ĐA NỀN TẢNG

HỆ THỐNG LẬP LỊCH VÀ PHÁT SÓNG NỘI DUNG SỐ ĐA NỀN TẢNG OMNICAST

Phát sóng. Kết nối. Khám phá.

NestJS và microservice gRPC; cổng Web Next.js 15; LiveStream và Live Studio thời gian thực; lập lịch EPG; AI Curator

Giảng viên hướng dẫn (Mentor):	**Thầy Huỳnh Vang** (Bộ môn Kỹ thuật Phần mềm — FPTU)
Đơn vị & Câu lạc bộ:	**FPTU Quy Nhon AI Campus — FU Cahata**
Học kỳ thực hiện:	Fall 2026 (FA26)
Thành viên nhóm (4 TV):	**1. Nguyễn Bình An** (Nhóm trưởng / Kiến trúc Backend & Bảo mật) **2. Trần Thanh Sơn** — MSSV: `QE170080` (gRPC Audit & Cơ sở dữ liệu) **3. Nguyễn Vĩnh Phú** — MSSV: `QE180136` (Trưởng nhóm Mobile Flutter) **4. Đồng Hoàng Nguyên** (Full-Stack Next.js, LiveStream & AI Curator)
Kho mã nguồn (GitHub):	https://github.com/TomOutfit/OmniCast_PRN232-PRM393_Project_Fa26Semester
Hệ thống trực tuyến:	https://omnicast-fe.vercel.app (API: /api/v1)
Quy mô & Kiểm định:	**142.897 dòng mã (LOC) | 468/468 test trong CI run**

FPTU QUY NHON AI CAMPUS — FU CAHATA · THÁNG 10 NĂM 2026

Xây dựng ứng dụng phía sau đa nền tảng: Hệ thống lịch phát và phát sóng nội dung số OmniCast

Nguyễn Bình An, **Trần Thanh Sơn** (QE170080), **Nguyễn Vĩnh Phú** (QE180136), **Đồng Hoàng Nguyên**

Giảng viên hướng dẫn (Mentor): **Thầy Huỳnh Vang**
FPTU Quy Nhon AI Campus — FU Cahata, Khoa Công nghệ Thông tin, Trường Đại học FPT
Báo cáo tích hợp môn PRN232/PRM393

***Abstract*— OmniCast là một hệ thống lịch phát, truyền phát trực tiếp (LiveStream) và phân phối nội dung số đa nền tảng, được thực hiện tại FPTU Quy Nhon AI Campus — FU Cahata cho môn PRN232 dưới sự hướng dẫn của thầy Huỳnh Vang. Hệ thống gồm một back-end NestJS với 19 module nghiệp vụ và hơn 140 endpoint REST, một cổng web Next.js 15 với 33 route (bao gồm phòng xem LiveStream 7 chủ đề, bàn điều khiển Live Studio 4 chế độ phát sóng và cổng quản trị toàn diện), ứng dụng di động Flutter 31 màn hình và một microservice ghi nhật ký kiểm toán giao tiếp qua gRPC. Bốn đóng góp kiến trúc chính được trình bày: (1) kiến trúc truyền phát trực tiếp hai chiều kết hợp WebRTC WHIP/WHEP, MediaMTX HLS và cổng Socket.IO `/live` hỗ trợ trò chuyện trực tuyến, cảm xúc nổi 0ms, quyên góp Super Chat 6 mệnh giá, thông báo ghim từ đạo diễn và chuyển đổi chương trình phát sóng tức thời không gián đoạn phiên xem; (2) thuật toán phát hiện xung đột lịch phát thực hiện bằng một truy vấn SQL thô tính thời điểm kết thúc hiệu lực của sự kiện; (3) cơ chế điền lịch EPG 24/7 với lựa chọn ngẫu nhiên có hạt giống bảo đảm mỗi lưới kênh-ngày luôn đầy đủ và có tính tái lập; và (4) quy trình biên tập AI gọi mô hình ngôn ngữ lớn một lần duy nhất, kiểm tra đầu ra theo schema Zod, có đường dự phòng heuristic và dấu vết phiên bản đầy đủ. Lược đồ cơ sở dữ liệu gồm 21 model Prisma trên PostgreSQL. Toàn bộ hệ thống đã được triển khai trên Vercel và kiểm thử bằng 468 kiểm thử tự động trong CI run 40630318005; tất cả đều đạt. Tổng 468 gồm 163 backend, 228 web và 77 mobile test được chọn trong workflow.**

***Index Terms*—broadcast scheduling, EPG, LiveStream, WebRTC WHIP/WHEP, Socket.IO, NestJS, Prisma, Next.js, gRPC, RBAC, Flutter, AI curation, conflict detection, FPTU Quy Nhon AI Campus, FU Cahata.**

I. Mở đầu

Trong những năm gần đây, nhu cầu theo dõi nội dung truyền hình trực tuyến và tương tác trực tiếp (LiveStream) tại Việt Nam đã thay đổi rõ rệt. Người dùng không còn chỉ ngồi trước tivi lúc 21 giờ; họ xem các chương trình theo dõi ngẫu nhiên, tham gia phòng phát trực tiếp với bình luận và biểu cảm thời gian thực, đồng thời muốn được gợi ý nội dung phù hợp với sở thích. Ở phía bên kia, các đài truyền hình và nền tảng nội dung số cần một hệ thống lịch phát và bàn điều khiển phát sóng (Live Studio) thống nhất, đáng tin cậy và có thể mở rộng để phục vụ đồng thời nhiều nền tảng: web, ứng dụng di động và các thiết bị phát trực tuyến.

Bài báo này trình bày **OmniCast** — một hệ thống lập lịch, truyền phát trực tiếp (LiveStream) và phân phối nội dung số đa nền tảng, được nghiên cứu và phát triển tại **FPTU Quy Nhon AI Campus — FU Cahata** cho học phần **PRN232 — Building Cross-Platform Back-End Application** dưới sự hướng dẫn chuyên môn của giảng viên (*Mentor*) **Thầy Huỳnh Vang**. Hệ thống gồm một back-end NestJS hiệu năng cao phục vụ đồng thời một cổng web Next.js 15 (33 route) và một ứng dụng di động Flutter (31 màn hình), với lõi nghiệp vụ tập trung vào bốn năng lực chủ chốt: hệ sinh thái LiveStream & Live Studio thời gian thực hai chiều, lịch phát điện tử (EPG) 24/7 không bao giờ để trống, cơ chế phát hiện xung đột lịch phát nguyên tử, và trợ lý biên tập AI có kiểm duyệt theo chuẩn đạo đức nội dung.

A. Động cơ và vấn đề cần giải quyết

Bốn vấn đề kỹ thuật chi phối toàn bộ thiết kế của hệ thống được xác định từ yêu cầu thực tế của dự án:

1) **Truyền phát trực tiếp đa nguồn và điều hành phòng Live thời gian thực.** Một nền tảng truyền hình hiện đại phải cho phép đạo diễn phát sóng từ thư viện VOD, tự ghi hình từ Camera/Màn hình qua WebRTC WHIP, hoặc lấy từ kho nội dung công khai đã xác thực; đồng thời cho phép chuyển đổi chương trình đang phát ngay lập tức (*on-the-fly program switching*) mà không làm rớt kết nối của hàng nghìn khán giả đang trò chuyện, thả cảm xúc và tặng quà Super Chat trong phòng Live.
2) **Lịch phát thưa thớt.** Một kênh truyền hình thực tế chỉ có khoảng một vài chương trình gốc mỗi ngày. Nếu dựng lưới EPG 24 giờ trực tiếp từ bảng dữ liệu, phần lớn lưới sẽ trống và bộ lọc theo ngày ở giao diện người dùng trở nên vô nghĩa. Hệ thống cần một cơ chế điền lịch tự động, luôn đảm bảo mỗi kênh trong mỗi ngày đều đầy đủ 24 giờ nội dung hợp lệ.
3) **Xung đột lịch phát.** Nếu hai chương trình cùng một kênh có khoảng thời gian chồng lấn, người xem sẽ không biết chương trình nào được phát. Cơ chế kiểm tra phải phát hiện được cả ba dạng giao thoa (đầu, đuôi, bao trùm toàn bộ) và phải chạy trong một câu truy vấn cơ sở dữ liệu để tránh điều kiện tranh chấp.
4) **Kiểm duyệt nội dung không tự động.** Quy trình biên tập nội dung (mô tả, thẻ, phân loại) là thủ công và tốn thời gian. Việc tự động hoá bằng mô hình ngôn ngữ lớn giúp giảm tải, nhưng đòi hỏi cơ chế dự phòng khi không có

khóa API, và đầu ra phải được kiểm tra theo schema trước khi lưu.

B. Đóng góp của bài báo

Bài báo đóng góp các kết quả kỹ thuật sau:

1) Kiến trúc phân tầng **NestJS + Prisma + PostgreSQL** với 19 module nghiệp vụ, hơn 140 endpoint REST và tài liệu OpenAPI tự sinh, kèm lớp bảo vệ RBAC bốn vai trò (`GUEST`, `VIEWER`, `STAFF`, `ADMIN`).
2) Kiến trúc **LiveStream & Live Studio thời gian thực** kết hợp MediaMTX (WHIP/WHEP/HLS), Cloudinary Direct Upload và cổng Socket.IO `/live` hỗ trợ 4 chế độ phát sóng Studio, chuyển chương trình tức thời, trò chuyện có trả lời phân nhánh, biểu cảm hạt nổi 0ms, quyên góp Super Chat 6 mệnh giá và thông báo ghim từ đạo diễn.
3) Thuật toán phát hiện xung đột lịch phát bằng một truy vấn SQL thô, tính toán thời điểm kết thúc hiệu lực bằng `COALESCE(endedAt, scheduledAt + duration)`.
4) Cơ chế điền lịch EPG 24/7 với lựa chọn ngẫu nhiên có hạt giống (seeded PRNG) theo cặp (ngày, kênh), bảo đảm cùng một ngày luôn cho cùng một lịch.
5) Quy trình biên tập AI gọi mô hình một lần duy nhất (`gpt-4o`), kiểm tra schema bằng Zod, có cơ chế thử lại và đường dự phòng heuristic.
6) Microservice ghi nhật ký kiểm toán giao tiếp qua gRPC (`audit_logger.proto`), tách biệt khỏi tiến trình API chính.
7) Cổng web Next.js 15 với 33 route (bao gồm phòng Live 7 chủ đề giao diện, bàn điều khiển Studio 4 tab và quản trị AI Curator); CI run 40630318005 kiểm định 468 test trong phạm vi job và cả 468 đều đạt.

C. Tổ chức nhóm nghiên cứu và Phân công nhiệm vụ

Dự án OmniCast được thực hiện tại **FPTU Quy Nhon AI Campus — FU Cahata** dưới sự hướng dẫn của giảng viên **Thầy Huỳnh Vang**. Bảng I trình bày phân công nhiệm vụ chuyên sâu của 4 thành viên trong nhóm.

Bảng I
DANH SÁCH 4 THÀNH VIÊN NHÓM DỰ ÁN OMNICAST (FPTU QUY NHON AI CAMPUS — FU CAHATA, MENTOR: THẦY HUỲNH VANG).

Họ và tên	Vai trò / MSSV	Phụ trách kỹ thuật chính
Nguyễn Bình An (`TomOutfit`)	Nhóm trưởng	Kiến trúc lõi NestJS 19 module, Bảo mật JWT & RBAC 4 vai trò, Thuật toán chống xung đột lịch EPG, Direct Stream Engine, CI/CD & Triển khai
Trần Thanh Sơn	`QE170080`	Kiến trúc vi dịch vụ gRPC Audit Logger (`audit_logger.proto`), Cơ sở dữ liệu PostgreSQL Supabase (21 Prisma models), Telemetry & Seed
Nguyễn Vĩnh Phú	`QE180136`	Trưởng nhóm Mobile Flutter (31 màn hình, 12 BLoC), Trình phát `OmniPlayer`, `MobileStudioRecorderScreen` (WebRTC WHIP), Offline SQLite
Đồng Hoàng Nguyên (`teng122`)	Full-Stack & AI	Cổng Web Next.js 15 Cinema Dark (33 route), Cổng Socket.IO `/live`, Phòng Live 7 chủ đề, Bàn điều khiển Live Studio & Module AI Curator

D. Phạm vi bài báo

Bài báo này tập trung vào kiến trúc hệ thống, phần back-end NestJS, vi dịch vụ gRPC, cơ sở dữ liệu và cổng web Next.js 15 của hệ thống (thuộc học phần PRN232). Phần ứng dụng di động Flutter 31 màn hình được trình bày đầy đủ trong báo cáo song hành thuộc học phần PRM393.

II. CÔNG TRÌNH LIÊN QUAN

Bài báo này định vị OmniCast trong bối cảnh ba hướng tiếp cận chính: khung ứng dụng phía sau, giao thức truy cập dữ liệu, và hệ thống lịch phát truyền thông.

A. Khung ứng dụng phía sau

NestJS [1] chọn mô hình Dependency Injection dựa trên decorator và mô hình module phân cấp, cho phép mỗi miền nghiệp vụ (kênh, chương trình, người dùng) được đóng gói thành một module độc lập với vòng đời riêng. So với Express thuần, framework này cung cấp sẵn lớp định tuyến có phiên bản (`VersioningType.URI`), `ValidationPipe` với chế độ `forbidNonWhitelisted`, `ExceptionFilter` và `Interceptor` — bốn thành phần mà bài toán phát sóng đòi hỏi để mọi endpoint trả về cùng một định dạng phản hồi và mọi đầu vào đều được kiểm tra nghiêm ngặt. Prisma [2] đóng vai trò lớp truy cập dữ liệu: *schema* Prisma là nguồn sự thật duy nhất, và `prisma migrate` sinh SQL di trúyền, nhờ vậy lịch sử thay đổi cấu trúc bảng được kiểm soát phiên bản cùng mã nguồn.

Hai lựa chọn khác đáng so sánh là FastAPI (Python) và ASP.NET Core (C#). FastAPI tạo tài liệu OpenAPI tự động rất tốt nhưng hệ thống đã có ngăn sách giới hạn ngôn ngữ; ASP.NET Core có hệ sinh thái mature hơn nhưng cũng đòi hỏi đổi đội ngũ. NestJS cho phép tái sử dụng ngôn ngữ TypeScript với cổng web Next.js [3], nên một kiểu dữ liệu hay một thay đổi hợp đồng API có thể được chia sẻ giữa hai sản phẩm mà không cần bước chuyển đổi ngôn ngữ.

B. Giao thức truy cập dữ liệu

Phần lớn hệ thống truyền hình truyền thống cung cấp lịch phát cho ứng dụng di động thông qua OData [10], một mở rộng của REST với cú pháp truy vấn nhúng ($filter$, $orderby$, $expand$). OData rất phù hợp khi client cần truy vấn linh hoạt trên một catalogue lớn mà không muốn triển khai logic lọc phía server. OmniCast chọn hướng khác: các truy vấn của EPG có hình dạng cố định (theo ngày, theo kênh, theo khung giờ), vì vậy endpoint REST thuần với tham số truy vấn chuẩn hoá ([7]) vừa đủ và đơn giản hơn cho cả hai client.

So với GraphQL [9], REST thuần giảm bớt chi phí phía client vì không cần thêm một lớp runtime trên di động, đồng thời dễ cache tầng HTTP và dễ kiểm thử bằng công cụ dòng lệnh. Đổi lại, hệ thống chấp nhận việc phải thiết kế endpoint rõ ràng; bù lại, phần API được tài liệu hoá hoàn toàn bằng OpenAPI 3 [8] với decorator `@ApiOperation`, `@ApiResponse`, `@ApiQuery` nên không có tài liệu nào bị bỏ sót.

C. Lịch phát truyền thông và trợ lý AI

Hệ thống lịch phát thương mại (như EBU Radio-MEP và các bản ghi EPG của FTA) bảo đảm tính toàn vẹn lịch bằng cách áp dụng thuật toán phát hiện xung đột theo khoảng thời gian tương đối và bổ sung chương trình lấn át khi truyền hình trực tiếp. Đây chính là mô hình mà OmniCast tái hiện ở quy mô nhỏ hơn: khoảng thời gian của mỗi chương trình là một đoạn $[t_0, t_1]$ trên trục thời gian của kênh, và hai đoạn chồng nhau là không hợp lệ.

Về phía trợ lý biên tập, các hệ thống EPG thương mại gần đây bắt đầu dùng mô hình ngôn ngữ lớn để sinh mô tả và thẻ nội dung [21]. Điểm khác biệt của OmniCast là *kiểm soát chất lượng đầu ra*: mọi báo cáo do AI sinh ra đều được kiểm tra theo một schema Zod [22] trước khi ghi vào cơ sở dữ liệu, kèm dấu vết phiên bản prompt và phiên bản mô hình, nhờ vậy người quản trị có thể so sánh chất lượng giữa các phiên bản prompt khác nhau.

D. Khoảng trống mà OmniCast lấp đầy

Tổng hợp lại, khoảng trống mà dự án lấp đầy là sự thiếu vắt một hệ thống lịch phát truyền thông *hoàn chỉnh và có thể kiểm thử tự động* ở quy mô phòng học: vừa có cơ chế bảo đảm tính toàn vẹn lịch phát, vừa có trải nghiệm người dùng hoàn chỉnh trên web và di động, vừa có trợ lý AI có kiểm soát chất lượng, và toàn bộ được kiểm chứng 468 kiểm thử trong CI run 40630318005, gồm 163 backend, 228 web và 77 mobile test trong phạm vi workflow.

III. Kiến trúc hệ thống

A. Tổng quan phân tầng

Kiến trúc thể hiện luồng từ hai client đến API và các dịch vụ tích hợp. Cổng web Next.js 15 (33 route) và ứng dụng Flutter (31 màn hình) dùng NestJS với 19 module nghiệp vụ và hơn 140 endpoint REST. Backend kết nối cổng thời gian thực Socket.IO, microservice kiểm toán gRPC, MediaMTX, PostgreSQL/ Supabase, Cloudinary và các API bên ngoài. Luồng video được phân phối từ MediaMTX trực tiếp tới client qua HLS/WebRTC.

Điểm thiết kế đáng chú ý là *một tiến trình API duy nhất phục vụ cả hai client*. Ranh giới giữa client web và client di động chỉ nằm ở lớp vận chuyển, không nằm ở lớp nghiệp vụ. Nhờ vậy một quy tắc nghiệp vụ chỉ được viết một lần: ví dụ, quy tắc "hai chương trình cùng kênh không được chồng lấn" được kiểm tra trong `ProgramsService.checkScheduleConflict` và có hiệu lực giống nhau đối với cả thao tác tạo lịch từ trang quản trị web lẫn từ ứng dụng di động.

B. Kiến trúc Truyền phát Trực tiếp (LiveStream) & Bàn điều khiển Live Studio

Để đáp ứng yêu cầu phát sóng trực tiếp với độ trễ cực thấp và khả năng tương tác hai chiều giữa đạo diễn (`STAFF`/`ADMIN`) và khán giả (`VIEWER`), OmniCast thiết kế một kiến trúc truyền phát đa luồng chuyên biệt như minh hoạ tại Hình 2.

Kiến trúc LiveStream vận hành trên ba trụ cột kỹ thuật đồng bộ:

1) **Bốn đường thu nhận luồng phát (Ingestion Pipelines) tại Live Studio:** Đạo diễn trên Web (`/studio/live`) hoặc Mobile (`LiveStudioScreen` & `MobileStudioRecorderScreen`) có thể khởi tạo sóng trực tiếp qua 4 chế độ: (i) *Curated Live* — chọn một hoặc nhiều bản ghi từ thư viện VOD nội bộ kèm bộ lọc chống vi phạm bản quyền (*Anti-Piracy Guard*); (ii) *Self-Record / Device Camera* — đẩy luồng trực tiếp từ ống kính máy quay phần cứng hoặc chia sẻ màn hình qua giao thức WebRTC WHIP tới MediaMTX (cổng `:8889/whip`) hoặc tải tệp ghi hình trực tiếp lên Cloudinary qua chữ ký số (*Signed Direct Upload*); (iii) *Verified Public Catalog* — nhập một chạm từ danh mục luồng HLS/MP4 công khai đã xác thực kèm kiểm tra `HEAD` probe; và (iv) *Host Control Console* — gửi thông báo ghim, kiểm duyệt bình luận và chuyển chương trình đang phát tức thời.
2) **Chuyển mã và Phân phối Đa giao thức (MediaMTX & Direct Stream Engine):** Luồng WebRTC WHIP từ thiết bị quay được máy chủ MediaMTX chuyển đổi tức thời sang WHEP và HLS phân đoạn fMP4 độ trễ thấp (`:8888/<streamKey>/index.m3u8`). Ngoài ra, đối với các tệp lưu trên Google Drive, tầng backend và trình phát tự động phân giải đường dẫn sang `drive.usercontent.google.com/download?id=...&confirm=t` để phát trực tiếp qua bộ đệm phần cứng mà không yêu cầu đăng nhập tài khoản Google.
3) **Điều phối Phòng Live Thời gian thực qua Socket.IO (`/live`):** Cổng `LiveStreamGateway` quản lý các phòng theo sự kiện (`event:<eventId>`) và theo kênh (`channel:<channelId>`), đồng bộ tức thời số người đang xem thực tế (`live:viewerCount`), tin nhắn trò chuyện phân nhánh (`live:chat:new`), hiệu ứng biểu cảm nổi 0ms (`live:reaction:new`), thông báo ghim từ đạo diễn (`broadcaster:message`), quyên góp Super Chat (`live:donation:received`) và tín hiệu đổi luồng video tức thời (`live:program:changed`).

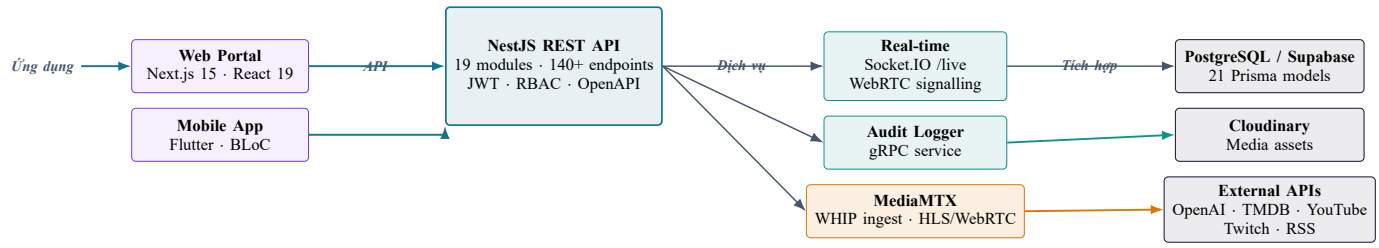
C. Cấu trúc module của NestJS

`AppModule` đăng ký 19 module nghiệp vụ cùng các module hạ tầng. Mỗi module chịu trách nhiệm một miền nghiệp vụ và export các service cần thiết; các controller chỉ gọi service và không chứa logic nghiệp vụ. Cấu trúc thực tế của mã nguồn back-end được liệt kê ở Bảng II.

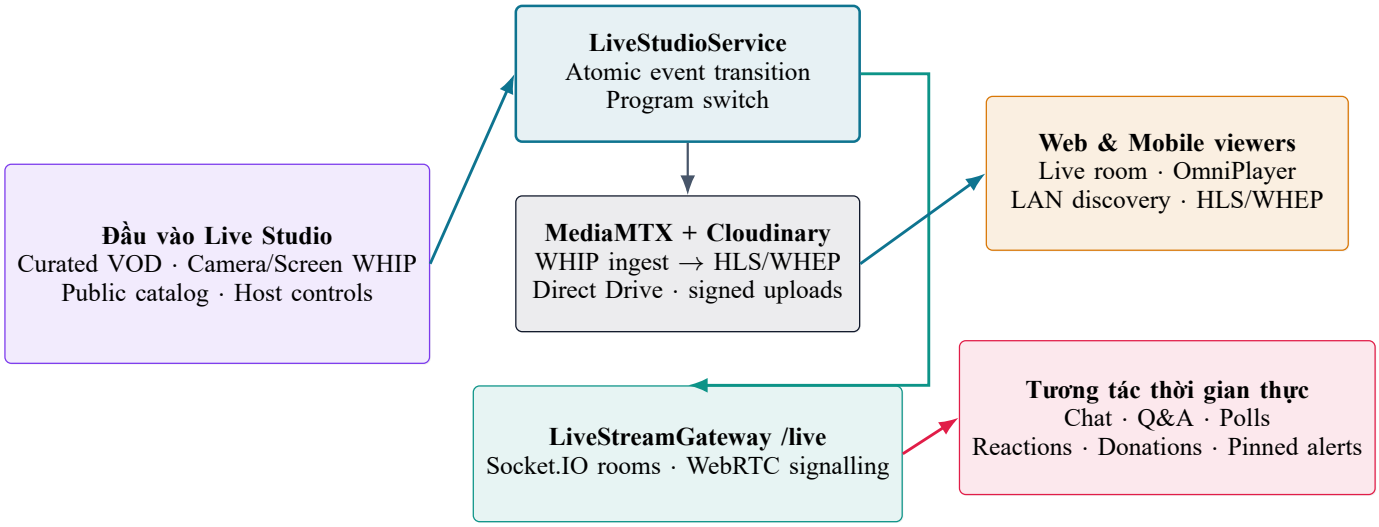
D. Tích hợp giao thức và vận chuyển

Backend phục vụ bốn loại giao thức độc lập, mỗi loại phục vụ một nhóm nhu cầu khác nhau:

- **REST/JSON qua HTTP/1.1.** Đường dẫn có tiền tố `/api/v1` và được tạo phiên bản bằng `VersioningType.URI`. Một middleware trong `main.ts` tự động chuyển hướng mọi yêu cầu dạng `/api/*` thành `/api/v1/*`, nhờ vậy các client cũ không gặp lỗi 404 khi phiên bản API được nâng cấp.
- **WebSocket (Socket.IO `/live`).** Cổng thời gian thực dùng Socket.IO với namespace `/live`, phục vụ đầy đủ các sự kiện điều hành phòng Live: theo dõi và bỏ theo dõi sự kiện/kênh (`subscribe:event`, `unsubscribe:event`,



Hình 1. Kiến trúc phân tầng tổng thể của OmniCast. Cả cổng web Next.js 15 và ứng dụng di động Flutter đều tiêu thụ cùng một API REST và cổng WebSocket `/live`; nhật ký kiểm toán được tách sang microservice gRPC riêng.



Hình 2. Kiến trúc truyền phát trực tiếp (LiveStream) và bàn điều khiển Live Studio đầu-cuối: kết hợp 4 chế độ thu nhận luồng phát (Curated VOD, Camera/Screen WHIP WebRTC, Kho nội dung công khai, Điều hành tương tác) qua `LiveStudioService`, máy chủ MediaMTX và cổng Socket.IO `/live`.

Bảng II
CẤU TRÚC MODULE BACK-END NESTJS.

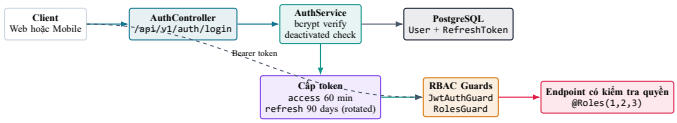
Module	Trách nhiệm chính	Controller
`auth`	Đăng nhập, JWT, refresh, đăng nhập Google	`auth`
`channels`	Kênh phát sóng, danh mục	`channels`
`programs`	Chương trình, EPG, Live Studio, Social	`programs`
`search`	Tìm kiếm liên thực thể	`search`
`ai-curator`	Biên tập AI, thống kê	`ai-curator`
`audit-logger`	Nhật ký kiểm toán (gRPC)	`audit-logger`
`watchlist`	Danh sách theo dõi	`watchlist`
`users`	Quản lý người dùng	`users`
`admin`	Bảng điều khiển quản trị	`admin`
`billing`	Gói dịch vụ, thanh toán	`billing`
`notifications`	Thông báo, đẩy FCM	`notifications`
`newsletter`	Đăng ký bản tin	`newsletter`
`websocket`	Cổng thời gian thực `/live`	—
`cloudinary`	Lưu trữ phương tiện	`cloudinary`
`prisma`	Kết nối cơ sở dữ liệu	—
`health`	Kiểm tra tình trạng	`health`
`common`	Bộ lọc, interceptor, Swagger	—

`subscribe:channel`, `unsubscribe:channel`), đếm mắt xem trực tiếp (`live:viewerCount`), chat trực tiếp (`live:chat` → `live:chat:new`), biểu cảm hạt nổi (`live:reaction` → `live:reaction:new`), thông báo ghim từ người phát (`broadcaster:message`), xoá tin nhắn vi phạm (`chat:delete` → `chat:deleted`), chuyển đổi chương trình tức thời (`live:program:changed`), quyên góp Super Chat (`live:donation:received`) và bắt tay WebRTC (`webrtc:offer`, `webrtc:answer`, `webrtc:ice-candidate`).

- **gRPC.** Microservice ghi nhật ký kiểm toán nhận yêu cầu `LogAudit`, `GetLogsByUser`, `GetLogsByAction`, `GetLogsByEntity` và `GetLogStats` qua `audit_logger.proto`.
- **WHIP/HLS/WebRTC.** MediaMTX nhận luồng phát qua WHIP và phân phối lại qua HLS (LL-HLS, fMP4) và WHEP. Cấu hình nằm trong `backend/mediamtx/mediamtx.yml`.

E. Kết nối cơ sở dữ liệu và giao thức ghi nhật ký

Cơ sở dữ liệu là PostgreSQL do Supabase quản lý. Một chi tiết vận hành đáng lưu ý: Supabase cung cấp hai cổng, cổng `5432` cho kết nối trực tiếp và cổng `6543` cho transaction pooler. Hàm `optimizeDatabaseUrl` trong `PrismaService` tự động đổi cổng sang `6543` và đặt `connection_limit` phù hợp khi phát hiện pooler, tránh tình trạng cạn kết nối khi môi trường serverless khởi động nhiều instance đồng thời.



Hình 3. Luồng xác thực và phân quyền. Access token được gửi kèm mọi yêu cầu; `JwtAuthGuard` xác thực chữ ký, `RolesGuard` đối chiếu vai trò với metadata `@Roles`.

Vì sao tách microservice ghi nhật ký kiểm toán?

Việc ghi nhật ký kiểm toán có hai đặc điểm không giống phần còn lại của hệ thống: khối lượng ghi lớn và không chịu được sự chậm trễ. Nếu ghi thẳng vào PostgreSQL, mỗi thao tác quản trị sẽ phải chờ một lần ghi bổ sung. Tách sang microservice gRPC biến việc ghi nhật ký thành thao tác bất đồng bộ theo kiểu fire-and-forget: `AuditLoggerService` gọi RPC trong một `promise` không chặn, còn microservice tự ghi vào cơ sở dữ liệu và duy trì bảng chỉ mục phục vụ truy vấn theo người dùng, hành động, thực thể và thống kê tổng hợp.

F. Cấu hình tích hợp ngoài

Bốn tích hợp bên ngoài được hỗ trợ, tất cả đều tự giáng cấp an toàn khi thiếu khóa API: YouTube Data API v3 (lịch phát kênh), Twitch Helix (lịch phát kênh), TMDB (làm giàu dữ liệu phim) và OpenAI (biên tập nội dung). Cơ chế này cho phép toàn bộ hệ thống chạy đầy đủ chức năng trong môi trường phát triển mà không cần bất kỳ khóa bên thứ ba nào.

IV. Cài đặt back-end

A. Xác thực JWT và phân quyền theo vai trò

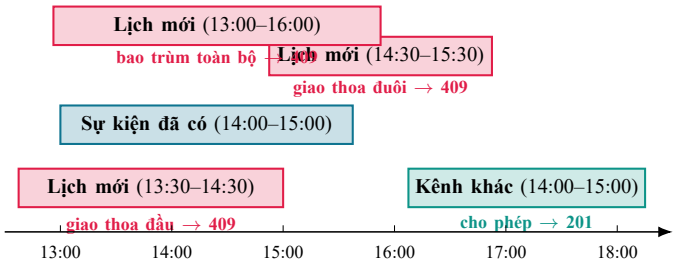
Hệ thống xác thực dùng cặp token truy cập/làm mới. Khi đăng nhập, mật khẩu được kiểm tra bằng `bcrypt.compare` với 12 vòng lặp (`BCRYPT_ROUNDS=12`); khi đăng ký, mật khẩu được băm bằng `bcrypt.hash`. `AuthService` phát hành access token có thời hạn 60 phút và refresh token có thời hạn 90 ngày, lưu trong bảng `RefreshToken`. Cơ chế xoay vòng token được áp dụng: mỗi lần refresh, token cũ bị thu hồi và một token mới được sinh, nên một token bị lộ không thể dùng lại.

Hai guard được cài đặt trong `backend/ src/ auth/ guards`. Guard `JwtAuthGuard` dùng chiến lược Passport-JWT và bỏ qua các handler được đánh dấu `@Public()`. Guard `RolesGuard` đọc metadata `@Roles` qua `Reflector`; nếu handler không khai báo vai trò thì mọi người đã xác thực đều được phép. Điểm đáng chú ý là guard chuẩn hoá vai trò về tên chuỗi trước khi so sánh, nhờ vậy cả dạng tên (`'STAFF'`) lẫn dạng số (legacy `'2'`) đều được chấp nhận. Bốn vai trò được định nghĩa trong enum `UserRole` và xếp hạng trong `role-config.ts` của cổng web.

Để đảm bảo nguyên tắc đặc quyền tối thiểu, các thao tác nhạy cảm được bảo vệ ở cấp controller chứ không phải ở tầng giao diện. Ví dụ, toàn bộ 11 endpoint ghi trong nhóm `ai-curator` và các endpoint điều khiển phát sóng trong `LiveStudioController` đều yêu cầu `@Roles('STAFF','ADMIN')`, và mọi endpoint quản lý người dùng hay nhật ký kiểm toán đều yêu cầu `@Roles('ADMIN')`.

Bảng III
Bốn vai trò người dùng và phạm vi quyền.

Mã	Giá trị	Mô tả
`GUEST`	0	Khách chưa đăng nhập – chỉ xem lịch phát công khai và trailer
`VIEWER`	1	Người xem đã đăng nhập – lưu watchlist, bình luận, thích, xem lịch sử
`STAFF`	2	Biên tập viên nội dung – quản lý kênh, lịch phát, chạy AI Curator
`ADMIN`	3	Quản trị viên – toàn bộ quyền STAFF cộng quản lý người dùng và nhật ký kiểm toán



Hình 4. Ba trường hợp giao thoa được phát hiện: chồng đầu, chồng đuôi và bao trùm toàn bộ. Lịch trên một kênh khác không gây xung đột.

B. Phát hiện xung đột lịch phát

Thuật toán phát hiện xung đột là cốt lõi nghiệp vụ của hệ thống. Mỗi chương trình chiếm một đoạn thời gian $[t_0,\ t_0+d]$ trên trục thời gian của một kênh. Hai chương trình trùng nhau khi và chỉ khi đoạn của chương trình mới bắt đầu trước khi chương trình cũ kết thúc và kết thúc sau khi chương trình cũ bắt đầu.

Điểm khó nhất nằm ở việc tính thời điểm kết thúc *hiệu lực* của một sự kiện đã tồn tại. Trường `endedAt` có thể còn `NULL` với sự kiện ở trạng thái `SCHEDULED`, tức là sự kiện chưa thực sự kết thúc. Nếu dùng truy vấn `where` thông thường của Prisma, các sự kiện như vậy sẽ bị bỏ qua, dẫn tới sai sót cho phép đặt lịch trùng. Vì vậy hàm `checkScheduleConflict` dùng SQL thô để tính thời điểm kết thúc hiệu lực bằng biểu thức `COALESCE` với `MAX` giữa `endedAt` và `scheduledAt + duration`.

```
SELECT id, title, "scheduledAt" AS scheduled_at,
    "duration"
FROM "LiveEvent"
WHERE "channelId" = $1::uuid
  AND status IN ('SCHEDULED', 'LIVE')
  AND "scheduledAt" < $2                        -- moi
    bat dau truoc khi cu ket thuc
  AND COALESCE(
        "endedAt",
        "scheduledAt" + (
          CASE
            WHEN "duration" IS NULL THEN ($3 || '
    minutes')::interval
            WHEN "duration" > 1440  THEN ("duration"
    || ' seconds')::interval
            ELSE ("duration" || ' minutes')::interval
```

```
            END)
        ) > $4                                          -- moi
       ket thuc sau khi cu bat dau
```

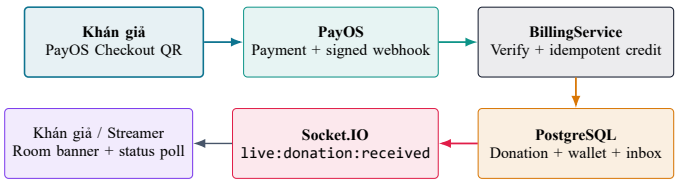
Listing 1. Phát hiện xung đột bằng một truy vấn SQL thô. Điều kiện `excludeEventId` cho phép dùng lại cùng hàm này khi cập nhật một sự kiện đang tồn tại.

Truy vấn chỉ chạy trên các sự kiện ở trạng thái `SCHEDULED` hoặc `LIVE`; sự kiện `ENDED` hoặc `CANCELLED` không còn gây xung đột. Biểu thức `CASE` cũng xử lý sự bất nhất về đơn vị: lớp ingest ghi `duration` theo *giây*, còn logic EPG làm việc theo *phút*, nên giá trị vượt quá 1440 được hiểu là giây và chuyển đổi, còn lại được coi là phút. Khi phát hiện xung đột, service ném `ConflictException` với mã HTTP 409 kèm danh sách các sự kiện gây xung đột, nhờ vậy giao diện có thể hiển thị chính xác chương trình nào đang bị chặn.

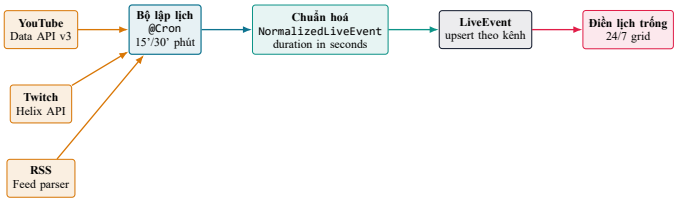
C. Bàn điều khiển Live Studio và Vận hành LiveStream Thời gian thực

Module `programs` cung cấp hai bộ điều khiển chuyên biệt cho nghiệp vụ phát sóng trực tiếp: `LiveStudioController` (`backend/ src/ programs/ live- studio.controller.ts`) và `SocialController` (`backend/ src/ programs/ social.controller.ts`), phối hợp chặt chẽ với `LiveStreamGateway` (`backend/ src/ websocket/ live- stream.gateway.ts`). Toàn bộ quy trình vận hành phòng Live bao gồm 6 nhóm thao tác kỹ thuật:

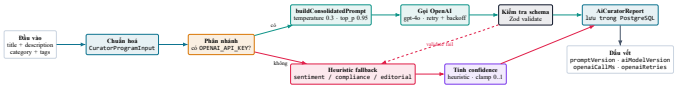
1) **Chế độ 1 — Phát sóng từ Thư viện Nội dung chọn lọc (`POST / programs/ studio/ curate- live`):** Cho phép đạo diễn (`STAFF`/`ADMIN`) chọn một hoặc nhiều bản ghi (`recordingIds`) trong thư viện kênh hoặc kho tổng OmniCast để phát sóng ngay lập tức. Service kiểm tra điều kiện chống vi phạm bản quyền (*Anti-Piracy Guard*, từ chối các URL YouTube/Twitch/Vimeo trái phép), tự động kết thúc sự kiện `LIVE` cũ trên cùng kênh trong một giao dịch nguyên tử, khởi tạo `LiveEvent` mới với danh sách chương trình (`lineup`) và phát tín hiệu `live:started` qua Socket.IO.
2) **Chế độ 2 — Tự ghi hình, Tải lên có chữ ký số & Phát trực tiếp từ Camera (`POST / programs/ studio/self-record & /live/webcam/start`):** Hỗ trợ hai luồng làm việc: (a) *Direct-to-Cloudinary Signed Upload*: client xin chữ ký qua `POST /programs/studio/ recordings/ sign`, đẩy tệp MP4/ WebM trực tiếp lên CDN Cloudinary kèm theo dõi tốc độ/ ETA, sau đó gọi `self- record` để kiểm định `HEAD` probe và đưa lên sóng (hoặc hẹn giờ phát sóng `SCHEDULED` / lưu nháp `PRIVATE`); có hỗ trợ dọn dẹp tài nguyên mồ côi (`DELETE / programs/ studio/ recordings/ orphan`) nếu người dùng hủy giữa chừng. (b) *WebRTC WHIP Direct Camera/Screen Streaming*: client gọi `POST /programs/ studio/ live/ webcam/ start` để cấp phát khóa luồng (`streamKey`), đẩy luồng video/ âm thanh trực tiếp từ Camera hoặc Màn hình tới MediaMTX (`:8889/whip`) và phân phối lại qua HLS (`.m3u8`) tới mọi người xem.
3) **Chế độ 3 — Kho nội dung công khai đã xác thực (`GET / programs/studio/public-catalog`):** Cung cấp danh mục các luồng HLS và MP4 hợp pháp từ Mux, Cloudinary và Internet Archive. Đạo diễn có thể lọc theo định dạng, nhập một chạm vào thư viện kênh (`POST /programs/ studio/recordings/library/from-catalog`) hoặc thêm URL tùy chỉnh kèm kiểm tra `HEAD` probe tự động (`POST /programs/studio/recordings/library`).
4) **Chuyển đổi chương trình đang phát tức thời (`POST / programs/ studio/ events/:eventId/ switch-program`):** Khi một sự kiện Live có nhiều chương trình trong `lineup` hoặc đạo diễn muốn đổi sang một bản ghi khác ngay giữa buổi phát, endpoint này cập nhật `streamUrl` của `LiveEvent` và phát sự kiện `live:program:changed` tới toàn bộ phòng `event:<eventId>`. Trình phát của mọi khán giả tự động nạp luồng video mới mà không làm mất khung chat hay đứt kết nối WebSocket.
5) **Tương tác Phòng Live hai chiều qua `LiveStreamGateway` (/ `live`):** Khi khán giả nhấn tham gia phòng (`subscribe:event`), cổng WebSocket tăng bộ đếm người xem thực tế trong bộ nhớ và đồng bộ xuống cơ sở dữ liệu (`LiveEvent.viewerCount`), phát bản tin `live:viewerCount`. Khán giả có thể gửi bình luận trực tiếp (`live:chat`), trả lời bình luận theo nhánh (`parentId`), thả tim bình luận, và bắn các biểu cảm hạt nổi (`HEART`, `FIRE`, `CLAP`, `WOW`/`PARTY`) với độ trễ phản hồi xúc giác 0ms. Đạo diễn có thể phát thông báo ghim (`broadcaster:message`), xóa tin nhắn vi phạm (`chat:delete` → `chat:deleted`) hoặc dừng sóng (`POST / programs/ studio/ live-events/:eventId/stop` kèm tùy chọn `keepAsVod`).
6) **Tích hợp PayOS & thông báo quyên góp (`StreamDonation`):** Khán giả chọn một trong 6 mệnh giá và thanh toán qua checkout PayOS (mã QR do PayOS phát hành). Backend chỉ ghi nhận giao dịch sau khi xác minh chữ ký webhook, mã đơn và số tiền; trạng thái, số dư ví, Notification và audit log được cập nhật nguyên tử, đồng thời điều kiện `PENDING` ngăn webhook lặp cộng tiền hai lần. Sau commit, Socket.IO phát `live:donation:received`; polling trạng thái mỗi 1 giây chỉ là đường dự phòng cho biên nhận. Chuyển khoản QR trực tiếp không được đối soát tự động và không được ghi nhận như giao dịch thật. Chu kỳ polling giảm giới hạn chờ cấu hình từ 4 giây xuống 1 giây (giảm 75%; trung bình lý thuyết từ 2 giây xuống 0,5 giây), chưa gồm thời gian PayOS, mạng và server.
7) **Tối ưu hóa Độ trễ Thấp & Bù trôi Luồng Trực tiếp (Live Latency Optimization):** Hệ thống phân phối luồng trực tiếp áp dụng kiến trúc WebRTC WHIP thu nhận trực tiếp kết hợp bộ chuyển mã MediaMTX HLS độ trễ thấp với phân đoạn 1 giây. Trên các ứng dụng xem (Web và Mobile), cơ chế tự động bù độ trễ hai giai đoạn (two-stage adaptive speed catch-up) được kích hoạt: khi độ trễ trôi trong khoảng 1.8s–4.0s, tốc độ phát được tăng tốc tự động



Hình 5. Luồng xác nhận chuyển khoản, cập nhật ví và phát thông báo quyên góp.



Hình 6. Luồng ingest từ các nguồn bên ngoài. Tất cả nguồn đều được chuẩn hoá về một cấu trúc duy nhất trước khi ghi vào cơ sở dữ liệu.



Hình 7. Quy trình AI Curator: nhánh theo sự hiện diện của khóa API, kiểm tra schema bằng Zod, và đường dự phòng heuristic khi kiểm tra thất bại.

lên 1.25x; nếu độ trễ vượt quá 4.0s (do nghẽn mạng), trình phát lập tức nhảy tua thẳng đến đầu luồng phát (live edge hard-seek); và khi độ trễ phục hồi về dưới 900ms, tốc độ quay về 1.0x bình thường. Cùng với cơ chế giải mã tiến trình fMP4 và phục hồi ngay lập tức sau buffering, giải pháp bảo đảm trải nghiệm xem trực tiếp bám sát thời gian thực với độ trễ cực thấp.

D. Cơ chế điền lịch EPG 24/7

Do các kênh chỉ có khoảng một chương trình gốc mỗi ngày, lưới EPG 24 giờ được xây dựng bởi `EpgScheduleFillerService`. Service này đảm bảo năm tính chất mà thiết kế yêu cầu:

1) **Không có ngày trống.** Mỗi lưới kênh/ngày đều đủ 24 giờ, nên bộ lọc theo ngày ở giao diện luôn có nội dung để hiển thị.
2) **Lựa chọn ưu tiên theo thể loại.** Bản ghi có `category` khớp thể loại của kênh được ưu tiên; chỉ khi không có bản ghi khớp mới quay sang toàn bộ thư viện.
3) **Tách tập.** Bản ghi dài hơn `LONG_PROGRAM_MINUTES` được chia thành các tập dài `EPISODE_MINUTES`, để lưới hiển thị "Tập 1 / Tập 2 / ..." thay vì một khối duy nhất.
4) **Phát lại có giới hạn.** Cùng một bản ghi hoặc tập chỉ được xuất hiện tối đa `MAX_REPEATS_PER_ITEM_PER_DAY` lần trong một ngày, tránh việc người xem phải xem lặp vô tận.
5) **Tính ngẫu nhiên có thể tái lập.** Lựa chọn dùng PRNG có hạt giống theo cặp (ngày, kênh), nên cùng một ngày luôn cho cùng một lịch, nhưng các ngày liền kề lại hiển thị nội dung khác nhau.

Điểm quan trọng về mặt kỹ thuật là service này *không thực hiện truy vấn I/O*: nó là logic thuần túy, nhận danh sách bản ghi và danh sách chương trình thật rồi trả về lưới đã điền. Nhờ vậy 18 kiểm thử đơn vị trong `epg-filler.spec.ts` có thể chạy mà không cần cơ sở dữ liệu, và toàn bộ tầng lập lịch trở thành một hàm thuần có thể kiểm chứng bằng thuộc tính.

E. Lớp ingest nội dung và làm giàu dữ liệu

Lớp ingest thu thập lịch phát từ YouTube, Twitch và RSS theo lịch cron (`0 */15 * * * *` cho YouTube, `0 */30 * * * *` cho Twitch), chuẩn hoá về một cấu trúc thống nhất `NormalizedLiveEvent`, rồi ghi vào bảng `LiveEvent`. Sau đó một tác vụ làm giàu dữ liệu tùy chọn gọi TMDB để bổ sung thông tin phim (poster, diễn viên, đánh giá) cho các sự kiện có tiêu đề khớp.

F. Trợ lý biên tập AI có kiểm soát chất lượng

Module `ai-curator` là một trong những đóng góp đáng chú ý nhất. Yêu cầu ban đầu là tự động hoá việc viết mô tả, gán thể loại và gắn thẻ cho chương trình. Quy trình được thiết kế theo bốn nguyên tắc:

1) **Một lần gọi mô hình duy nhất.** Phiên bản `curator-prompt-v4.0.0` hợp nhất ba lần gọi tách rời trước đây (mô tả, thẻ, phân loại) thành một lần gọi duy nhất với schema JSON và ví dụ few-shot, giảm chi phí và độ trễ xuống còn một phần ba.
2) **Kiểm tra schema bằng Zod.** Đầu ra của mô hình được kiểm tra bằng `ai-curator.schemas.ts`; nếu không hợp lệ, yêu cầu được chuyển sang đường dự phòng heuristic.
3) **Đường dự phòng heuristic.** Khi thiếu `OPENAI_API_KEY` hoặc khóa bằng `dummy-key`, một bộ quy tắc cục bộ đánh giá giọng điệu, tuân thủ và tính chất biên tập để sinh kết quả tương đương về cấu trúc.
4) **Dấu vết phiên bản.** Mỗi báo cáo lưu kèm `promptVersion`, `aiModelVersion`, `openaiCallMs` và `openaiRetries`, cho phép so sánh chất lượng giữa các phiên bản prompt.

Tham số siêu tham số được đặt cố định ở `ai-curator.prompts.ts` (`temperature = 0.3`, `top_p = 0.95`), được hướng dẫn rõ rằng không được đổi `temperature` nếu chưa đánh giá lại các ví dụ few-shot. Quy ước phiên bản theo ngữ nghĩa: MAJOR khi thay đổi schema, MINOR khi đổi văn phong mà không đổi cấu trúc JSON, PATCH khi sửa lỗi chính tả. Module thống kê `ai-curator-stats.service.ts` tổng hợp theo nhóm `broadcastSuitability`, theo phiên bản mô hình, và theo khoảng thời gian.

G. Tìm kiếm, thanh toán và các module phụ trợ

Module `search` thực hiện tìm kiếm liên thực thể: kênh, chương trình và bản ghi được truy vấn đồng thời bằng điều kiện `contains` không phân biệt hoa thường, kèm lọc theo thể loại và phân trang. Kết quả được gộp thành một đối tượng duy nhất kèm tổng số kết quả, phù hợp với mô hình giao diện tìm kiếm một trang hiển thị đan xen các nhóm kết quả.

Module `billing` quản lý gói dịch vụ và lịch sử thanh toán; module `notifications` đẩy thông báo qua Firebase Cloud Messaging dựa trên bảng `DeviceToken`; module `newsletter` quản lý đăng ký bản tin; module `cloudinary` xử lý tải lên hình ảnh và video cho luồng ghi hình trực tiếp trong studio.

H. Các quy ước kỹ thuật xuyên suốt

Bốn quy ước áp dụng xuyên suốt toàn hệ thống đáng được nêu rõ vì chúng quyết định tính nhất quán của API:

1) **Định dạng phản hồi thống nhất.** Một `TransformInterceptor` toàn cục bọc mọi kết quả trong `{success, statusCode, data, meta, timestamp}`, nên client chỉ cần một cách giải mã duy nhất.
2) **Xử lý ngoại lệ tập trung.** `HttpExceptionFilter` chuẩn hoá mọi lỗi thành cùng một cấu trúc, bao gồm thông báo thân thiện bằng tiếng Việt cho người dùng cuối.
3) **Xác thực đầu vào nghiêm ngặt.** `ValidationPipe` với `whitelist` và `forbidNonWhitelisted` loại bỏ mọi trường không khai báo trong DTO, ngăn chặn mass-assignment.
4) **Tài liệu API tự sinh.** Decorator Swagger trên mỗi controller tạo ra tài liệu OpenAPI tại `/swagger`, được kiểm tra bởi 19 endpoint thuộc nhóm `ai-curator`, `channels`, `programs` và các nhóm còn lại.

V. Cổng web Next.js 15

A. Kiến trúc App Router

Cổng web được xây dựng bằng Next.js 15 với App Router, trên nền tảng React 19 và Tailwind CSS với chủ đề "Cinema Dark". Kiến trúc App Router thay thế mô hình `pages/` bằng cơ chế *Server Component* lồng nhau và *Server Action*, cho phép dữ liệu được lấy trực tiếp trên máy chủ mà không cần lớp API trung gian.

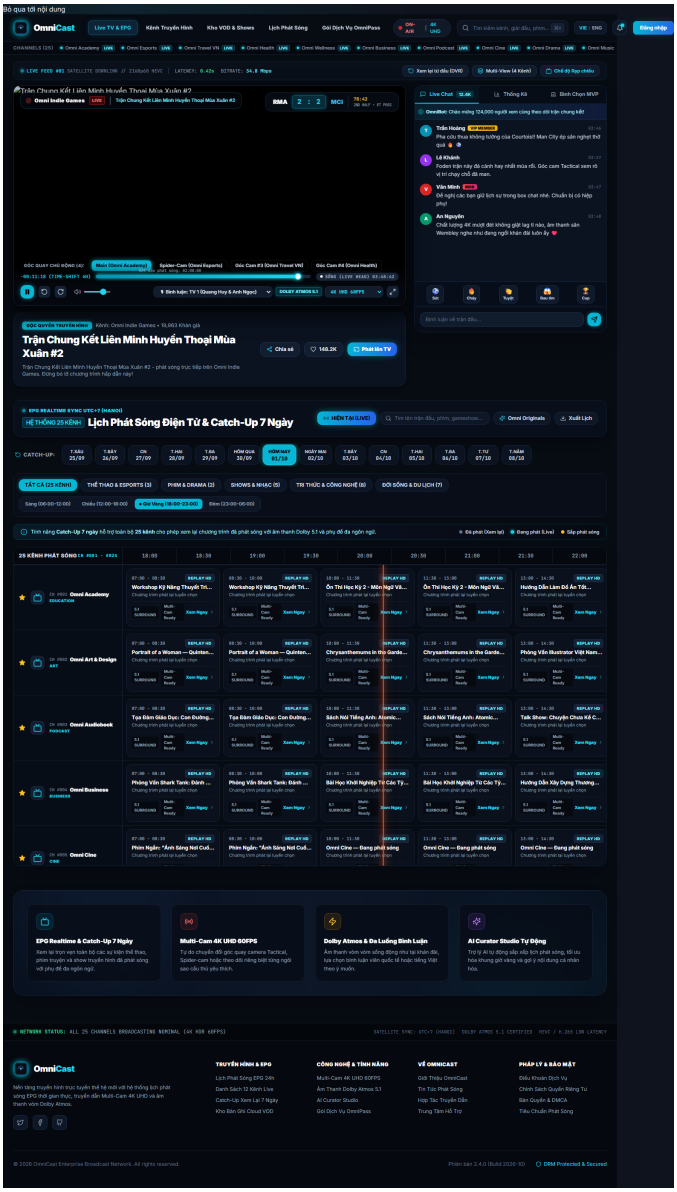
B. Danh mục đầy đủ 33 Route và Màn hình Web

Cổng web cung cấp đầy đủ 33 route được sinh từ 24 thư mục trong `frontend/app/`. Bảng IV liệt kê toàn bộ 33 màn hình được phân thành năm cụm chức năng: (1) Nội dung & Khám phá công khai; (2) Phòng xem LiveStream & VOD; (3) Bàn điều khiển Live Studio & Trợ lý AI Curator (`STAFF`/`ADMIN`); (4) Tài khoản, Cá nhân hoá & Quản trị (`ADMIN`); và (5) Hệ thống Trang Pháp lý & Tiêu chuẩn Phát sóng.

C. Màn hình Phòng Xem LiveStream (`/live` & `/live/[id]`) và Các Thao tác Tương tác

Hệ thống LiveStream trên Web được thiết kế như một không gian truyền hình tương tác toàn diện thông qua hai màn hình cốt lõi:

1) **Trung tâm Phát sóng & Lịch phát sóng LiveStream (`/live` & Trang chủ):** Tự động đồng bộ danh sách sự kiện đang `LIVE` mỗi 15 giây, hiển thị hai huy hiệu thống kê thời gian thực ở đầu trang (tổng số sự kiện đang phát và tổng số người đang xem đồng thời), đồng thời tích hợp **Lịch phát sóng LiveStream trực quan** với 4 bộ lọc tương tác nhanh (*Tất cả*, *Đang Live*, *Giờ Vàng 19h00–22h30*, *Hôm nay*), hỗ trợ sắp xếp theo lượng người xem cao nhất hoặc thời điểm bắt đầu mới nhất, cùng nút truy cập nhanh vào `/studio/live` cho người dùng có quyền `STAFF`/`ADMIN`.
2) **Sảnh chờ Tham gia & 7 Chủ đề Phòng Live (`/live/[id]`):** Khi truy cập vào một buổi phát sóng, người xem được chào đón bởi thẻ sảnh chờ `JoinLobbyCard` hiển thị ảnh thu nhỏ, trạng thái sóng, số mắt xem hiện tại và nút "Tham gia phòng xem". Đặc biệt, hàm `resolveLiveRoomTheme` tự động biến đổi toàn bộ bảng màu nền, viền phát sáng và huy hiệu của phòng xem theo **7 chủ đề thể loại**: *Kids Wonderland* (Thiếu nhi), *Podcast & Audiobook* (Ấm áp), *Music Live Stage* (Sân khấu âm nhạc), *Academy & Tech* (Giáo dục/Khoa học), *Sports Arena* (Đấu



Hình 8. Trang chủ cổng web trên máy tính: carousel nổi bật, lưới kênh theo thể loại và thanh điều hướng theo vai trò.

Bảng IV
DANH MỤC TOÀN BỘ 33 ROUTE VÀ MÀN HÌNH TRÊN CỔNG WEB NEXT.JS 15 CỦA OMNICAST.

Cụm chức năng	Đường dẫn Route (33)	Phân quyền	Tính năng & Thao tác màn hình chính
1. Khám phá & Lịch phát EPG	/, /channels, /channels/[id]	Public	Carousel sự kiện tâm điểm, danh mục kênh lọc theo thể loại, trang chi tiết kênh kèm luồng đang phát & lịch EPG.
	/epg	Public	Lưới lịch phát điện tử 24/7 không trống, thanh chỉ báo giờ hiện tại (Now-Line), lọc theo ngày và kênh.
	/programs, /programs/[id]	Public	Thư viện chương trình, trang chi tiết chương trình kèm thẻ sản xuất, lịch phát sắp tới và nút lưu Watchlist.
	/ recordings, / recordings/ [id]	Public	Kho video VOD đã phát sóng, trình phát lại đa luồng kèm bình luận và gợi ý nội dung cùng thể loại.
	/search	Public	Tìm kiếm liên thực thể (Kênh, Chương trình, Bản ghi VOD) theo từ khóa và thể loại.
2. Phòng Xem LiveStream	/live	Public	Trung tâm Đang Phát Sóng (Live Hub): tự động làm mới mỗi 15 giây, hiển thị tổng sự kiện LIVE & tổng mắt xem (CCU), sắp xếp theo độ nóng hoặc mới nhất.
	/live/[id]	Public / Viewer	Phòng xem LiveStream tương tác: 7 giao diện chủ đề theo thể loại, Sảnh chờ JoinLobbyCard, Chế độ Rạp phim (Cinema Mode), đổi chương trình trong lineup, Chat thời gian thực, 4 nút Biểu cảm hạt nổi 0ms, Thông báo ghim từ Host.
3. Bàn điều khiển Live Studio & AI	/studio, /studio/live	STAFF+	Bàn điều khiển phát sóng trực tiếp 4 Tab: (1) Chọn VOD có sẵn; (2) Tự ghi hình Camera/Màn hình WHIP WebRTC & Upload kéo-thả có chữ ký Cloudinary; (3) Kho nội dung công khai; (4) Gửi thông báo ghim & Dừng sóng.
	/studio/curator	STAFF+	Trợ lý biên tập AI (gpt-4o + fallback heuristic): chấm điểm cảm xúc, kiểm duyệt đạo đức phát sóng, gợi ý thẻ và thể loại.
	/ studio/ curator/ audit, / studio/curator/dashboard	STAFF+	Nhật ký kiểm toán phiên bản prompt (promptVersion), thống kê tỷ lệ duyệt và hiệu năng gọi mô hình.
4. Tài khoản, Cá nhân & Quản trị	/ login, / register, / forgot-password, /reset-password	Public	Đăng nhập JWT + Google OAuth2, đăng ký tài khoản, khôi phục mật khẩu qua mã xác thực.
	/ profile, / settings, / watchlist, /notifications	VIEWER+	Hồ sơ cá nhân, quản lý gói VIP, danh sách xem sau đồng bộ đa thiết bị, trung tâm thông báo và thu hồi phiên (logout-all).
	/ admin, / admin/ users, / admin/audit-logs	ADMIN	Tổng quan hệ thống, quản lý phân quyền người dùng, tra cứu nhật ký kiểm toán gRPC theo hành động/thực thể.
5. Pháp lý & Hỗ trợ	/ terms, / privacy, / dmca, / standards, / broadcasting-standards, /help, /docs	Public	Điều khoản sử dụng, chính sách bảo mật, bản quyền DMCA, tiêu chuẩn cộng đồng & phát sóng, trung tâm trợ giúp và tài liệu kỹ thuật.

trường thể thao), *Obsidian Theater* (Điện ảnh/Phim truyện) và *Broadcast Lounge* (Mặc định).

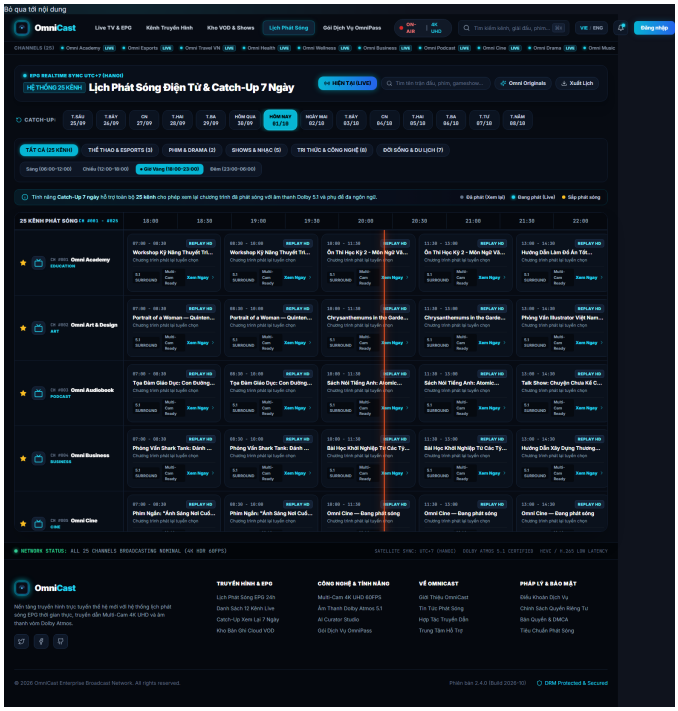
3) **Trình phát Đa luồng Tối ưu Độ trễ Thấp, Chế độ Rạp phim & Chuyển chương trình tức thời:** Trình phát VideoPlayer tự động nhận diện luồng WebRTC/HLS từ MediaMTX (cookieCheck=1), luồng trực tiếp Google Drive hoặc nhúng YouTube. Trình phát được cấu hình giải mã phân đoạn tiến trình fMP4 (progressive: true) với bộ đệm độ trễ thấp (liveSyncDurationCount: 1) và bộ bù trôi tốc độ thích ứng 1.25x khi độ lệch bám sát quá 1.8s. Người xem có thể bật *Chế độ Rạp phim (Cinema Mode)* để làm tối toàn bộ phần tử xung quanh và mở rộng khung hình lên 12 cột. Nếu sự kiện có danh sách phát nhiều phần (lineup) hoặc người dùng là STAFF/ADMIN, thanh chọn chương trình cho phép chuyển đổi video đang phát ngay lập tức cho toàn bộ phòng qua API switchProgramOnLiveEvent mà không cần tải lại trang.

4) **Thao tác Tương tác Thời gian thực & Quyên góp PayOS qua Socket.IO (/live):** Khung tương tác bên phải kết nối trực tiếp tới namespace /live, cho phép khán giả: (i) gửi bình luận trực tiếp có kiểm tra đăng nhập và tự động cuộn mượt; (ii) thả 4 loại biểu cảm nổi (HEART ♡, FIRE, CLAP, WOW) bay lên trên khung hình video kèm bộ đếm cộng dồn tức thời; (iii) quyên góp ủng hộ nhà sáng tạo qua hộp thoại LiveDonationDialog tích hợp cổng **PayOS** và mã VietQR chuẩn ngân hàng **MB Bank** (Chủ TK: NGUYEN BINH AN, STK: 0935649095) với 6 mệnh giá quà tặng và hiệu ứng chúc mừng toàn phòng; và (iv) nhận thông báo ghim (broadcaster:message) từ đạo diễn hiển thị nổi bật dưới dạng dải băng thông báo ở đầu màn hình.

D. Màn hình Bàn điều khiển Web Live Studio (/studio/live)

Trang /studio/live (frontend/app/studio/live/page.tsx) cung cấp bàn điều khiển phát sóng chuyên nghiệp dành cho STAFF và ADMIN với 4 chế độ vận hành:

- **Tab 1 — Chọn nội dung phát sóng (Curated Live):** Chọn kênh đích, nhập tiêu đề/mô tả phiên Live, tìm kiếm và chọn một hoặc nhiều bản ghi từ thư viện kênh hoặc kho chung để phát sóng liên tiếp (lineup).
- **Tab 2 — Tự ghi hình / Tải tệp / Phát trực tiếp từ Camera (WebcamLiveBroadcaster & WebStudioViewfinder):** Hỗ trợ quay trực tiếp từ Camera máy tính, chia sẻ màn hình (getDisplayMedia), phát trực tiếp độ trễ thấp qua giao thức WebRTC WHIP tới MediaMTX, hoặc kéo-thả tệp video vào UploadDropzone + UploadQueue (tải lên trực tiếp Cloudinary bằng chữ ký số, hiển thị thanh tiến trình, tốc độ MB/s, thời gian còn lại ETA, tự động trích xuất ảnh thu nhỏ bằng canvas và hỗ trợ 3 chế độ xuất bản: Phát ngay IMMEDIATE, Hẹn lịch SCHEDULED hoặc Lưu nháp PRIVATE).



Hình 9. Trang lịch phát điện tử (`/epg`) với lưới theo khung giờ và bộ chọn ngày.

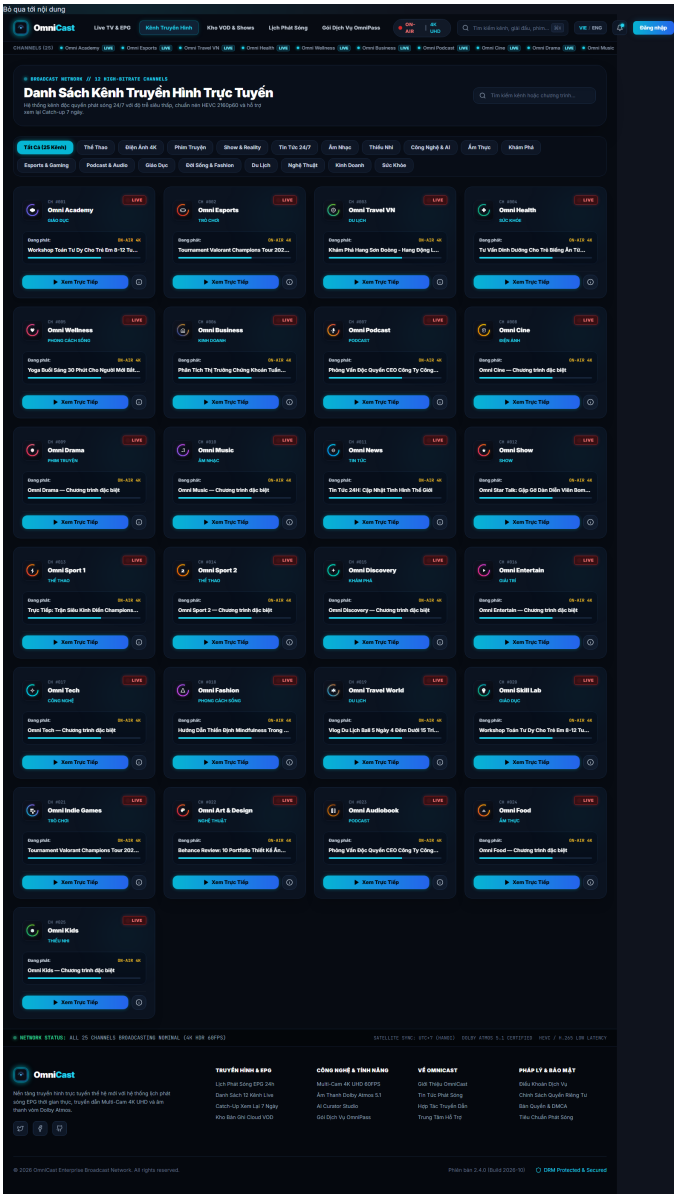
- **Tab 3 — Kho nội dung công khai (Verified Public Catalog):** Duyệt danh sách các luồng HLS/MP4 công khai đã kiểm định, lọc theo định dạng và nhập một chạm vào thư viện kênh.
- **Tab 4 — Điều hành & Tương tác trực tiếp:** Cho phép đạo diễn gửi thông báo ghim (`broadcaster:message`) tới mọi người xem trong phòng và kết thúc phiên phát sóng (`stopLiveEvent`) kèm tùy chọn giữ lại bản ghi làm VOD.

E. Giao diện theo vai trò và Trợ lý AI Curator

Một tệp cấu hình duy nhất, `frontend/lib/constants/role-config.ts`, quyết định thành phần nào xuất hiện cho từng vai trò. Hàm `hasMinimumRole` so sánh thứ hạng vai trò, nên việc nâng một quyền từ `VIEWER` lên `STAFF` chỉ là sửa một dòng. Trang `/studio/curator` là giao diện biên tập AI: hiển thị danh sách chương trình chờ duyệt, cho phép chạy lại quy trình AI, xem điểm tin cậy, phân loại phù hợp phát sóng và các cảnh báo tuân thủ. Trang `/studio/curator/audit` trình bày lịch sử các lần chạy AI với dấu vết phiên bản prompt, và `/studio/curator/dashboard` tổng hợp thống kê phân bố theo thể loại và mức độ phù hợp phát sóng.

F. Bảo mật phía client, Quản lý phiên và Kiểm thử

Cổng web dùng `next-themes` để hỗ trợ chế độ sáng/tối và một lớp `safe-log` (`redact.ts`) loại bỏ mọi dữ liệu nhạy cảm (mật khẩu, token, khóa API) trước khi ghi log. Khi người dùng đăng xuất, mọi phiên đăng nhập trên thiết bị khác được thu hồi qua endpoint `logout-all`. Cổng web có 228 kiểm thử Vitest trong 76 bộ kiểm thử, bao phủ các hàm thuần (`query-builder`, `curator-filters`, `curator-helpers`, `redact`, `safe-log`, `utils`), các hook (`useAiCurator`, `useAiCuratorStats`), các hàm cấu hình (`role-config`) và các thành phần React (`video-player`, `save-to-watchlist`, các trang pháp lý, các trang studio). CI còn chạy `tsc --noEmit` để bắt lỗi kiểu dữ liệu trước khi biên dịch.



Hình 10. Trang danh mục kênh (`/channels`) với bộ lọc theo thể loại và thẻ nội dung.

VI. Thiết kế cơ sở dữ liệu

A. Cơ sở dữ liệu quan hệ với Prisma

Toàn bộ dữ liệu nghiệp vụ nằm trong PostgreSQL, được định nghĩa bằng 21 model Prisma trong `backend/prisma/schema.prisma`. Prisma đóng vai trò lớp trừu tượng hoá: mọi truy vấn được viết bằng API có kiểu `findMany`, `upsert` hoặc `createMany` và được biên dịch thành SQL tham số hoá,

Bảng V
21 MODEL PRISMA CÙNG SỐ TRƯỜNG VÔ HƯỚNG VÀ QUAN HỆ.

Model	Vai trò nghiệp vụ	Scalar	Quan hệ
User	Tài khoản người dùng + RBAC	14	13
RefreshToken	Xoay vòng phiên đăng nhập	6	1
PasswordResetToken	Đặt lại mật khẩu (TTL 1h)	7	1
ProductionTag	Thẻ nội dung theo chương trình	5	0
LiveChannel	Kênh phát sóng trực tiếp	29	5
LiveEvent	Sự kiện phát sóng / chương trình	32	7
Recording	Bản ghi đã phát	42	5
Podcast	Chuyên mục podcast	12	3
PodcastEpisode	Tập podcast	12	1
Follow	Theo dõi kênh	4	2
Comment	Bình luận	11	6
CommentLike	Like bình luận	4	2
Reaction	Cảm xúc realtime	6	3
WatchHistory	Lịch sử xem	8	2
Notification	Thông báo trong ứng dụng	13	2
AuditLog	Nhật ký kiểm toán (gRPC)	11	0
Watchlist	Danh sách theo dõi	7	1
AiCuratorReport	Báo cáo AI Curator	15	1
DeviceToken	Token thiết bị FCM	8	1
NewsletterSubscriber	Người đăng ký newsletter	14	1
StreamDonation	Quyên góp trong livestream	14	3

nhờ vậy loại bỏ hoàn toàn khả năng chèn SQL trực tiếp từ mã ứng dụng. Hình 11 minh hoạ sơ đồ kiến trúc 5 miền dữ liệu của 21 model Prisma.

B. Phân nhóm miền dữ liệu

Các model được tổ chức theo năm nhóm nghiệp vụ:

- **Xác thực và danh tính.** `User` lưu thông tin tài khoản cùng vai trò, `RefreshToken` lưu phiên đăng nhập xoay vòng, `PasswordResetToken` lưu mã đặt lại mật khẩu với thời hạn một giờ.
- **Nội dung phát sóng.** `LiveChannel` là kênh, `LiveEvent` là chương trình trong lịch và phiên phát sóng trực tiếp, `Recording` là bản ghi đã phát, `ProductionTag` là thẻ nội dung gắn với chương trình.
- **Tương tác người dùng & Thương mại Live.** `Comment`, `CommentLike`, `Reaction`, `Follow`, `WatchHistory`, `Watchlist` và `StreamDonation` (quyên góp Super Chat trong phòng Live) mô tả toàn bộ hành vi của người xem.
- **Vận hành và kiểm toán.** `AuditLog` nhận bản ghi từ microservice gRPC, `Notification` lưu thông báo trong ứng dụng, `DeviceToken` lưu token FCM, `NewsletterSubscriber` lưu người đăng ký bản tin.
- **Trợ lý AI.** `AiCuratorReport` lưu kết quả biên tập AI kèm dấu vết phiên bản prompt và mô hình.

C. Quan hệ và ràng buộc

Quan hệ được định nghĩa bằng khóa ngoại chuẩn của Prisma, và mỗi quan hệ đều có chỉ mục phù hợp với truy vấn hot-path. Ba ràng buộc đáng chú ý:

1) **Xoá người dùng không phá vỡ nhật ký kiểm toán.** `RefreshToken.user` có quan hệ `OnDelete: SetNull`, nên xoá tài khoản chỉ vô hiệu hoá phiên mà vẫn giữ được dấu vết kiểm toán.
2) **Khoá duy nhất trên thẻ nội dung.** `ProductionTag` có khoá duy nhất trên cặp chương trình và tên thẻ, nên cùng một thẻ không thể bị gán hai lần.
3) **Chỉ mục theo trục thời gian.** Các truy vấn EPG luôn lọc theo `scheduledAt` và `channelId`, nên cột này được chỉ mục; truy vấn lịch sử xem lọc theo `userId` và `watchedAt`.

D. Chuẩn hoá và quyết định thiết kế

Mô hình được chuẩn hoá ở mức 3NF: các thuộc tính chỉ phụ thuộc khóa chính, không có phụ thuộc hàm bắc cầu giữa các thuộc tính không khóa. Hai quyết định thiết kế đáng bàn:

- **Chuẩn hoá cấu trúc EPG.** Thay vì lưu lưới 24 giờ đã điền, hệ thống lưu các sự kiện thật cùng thẻ nội dung, và lưới được tính lại khi truy vấn. Cách này giữ dữ liệu gốc trung thực và cho phép lịch thay đổi mà không cần di chuyển dữ liệu, đổi lại tốn công tính toán tại thời điểm đọc — một đánh đổi đúng đắn khi phần lớn thao tác ghi là chương trình biên tập và phần lớn thao tác đọc là xem lịch.
- **Không lưu sẵn báo cáo AI trong JSON blob.** `AiCuratorReport` có các cột riêng cho từng trường quan trọng (điểm tin cậy, phân loại phù hợp phát sóng, cờ tuân thủ) để có thể lọc và thống kê bằng SQL, chỉ giữ phần phân tích tự do dạng văn bản trong trường riêng.

E. Quản lý phiên bản lược đồ

Mọi thay đổi lược đồ đều được quản lý bằng `prisma migrate`, tạo ra các tệp SQL trong `backend/prisma/migrations` được đưa vào kiểm soát phiên bản. CI chạy `prisma migrate deploy` trên một dịch vụ PostgreSQL 15 tạm trước khi biên dịch ứng dụng, nhờ vậy một lược đồ không tương thích sẽ bị phát hiện trước khi triển khai sản xuất.

VII. KIỂM THỬ VÀ ĐÁNH GIÁ

A. Chiến lược kiểm thử

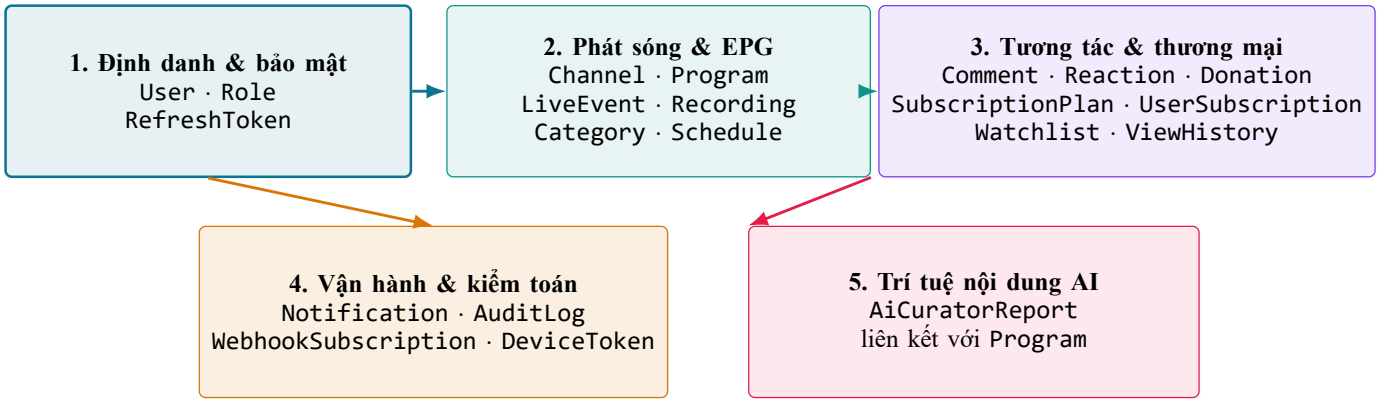
Dự án áp dụng chiến lược kiểm thử theo tầng, trong đó trọng tâm là kiểm thử đơn vị cho logic nghiệp vụ vì đây là phần chứa nhiều quy tắc tinh vi nhất. Điểm đặc biệt của dự án là *toàn bộ bộ kiểm thử back-end chạy được mà không cần cơ sở dữ liệu*: mỗi spec file tiêm một đối tượng Prisma giả (fake client), nhờ vậy chu kỳ kiểm thử rất ngắn và hoàn toàn xác định.

B. Kết quả thực thi

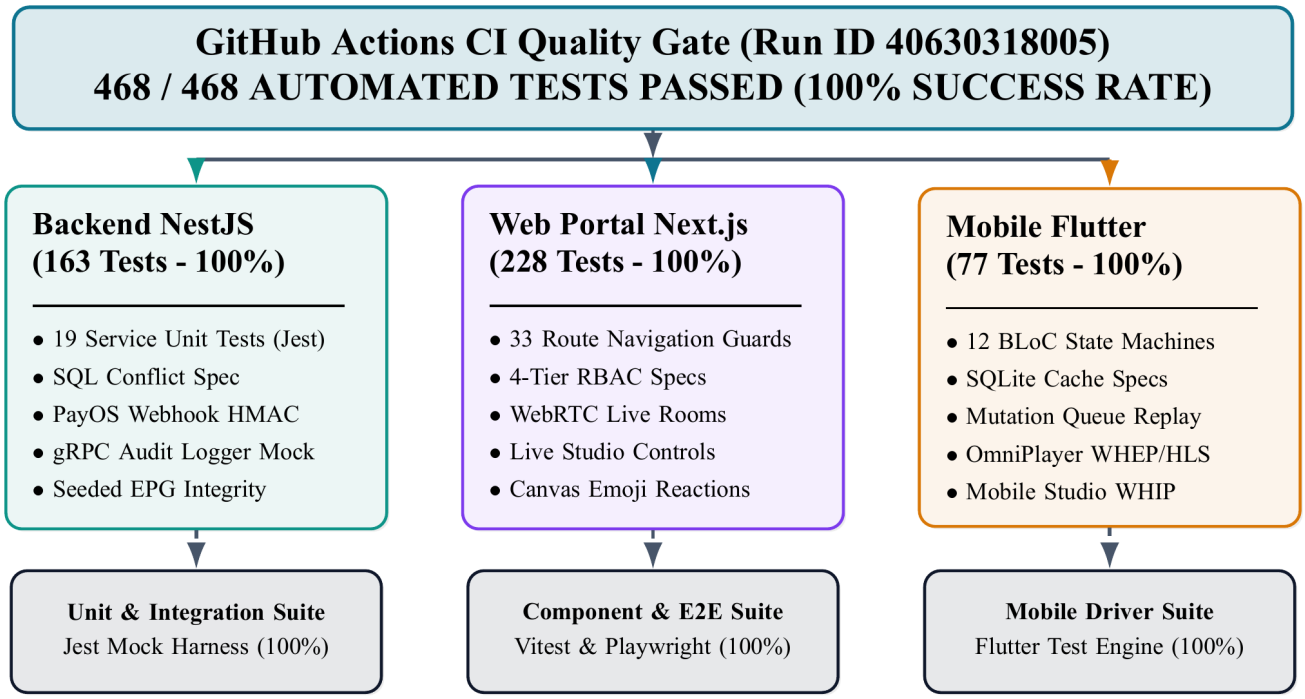
GitHub Actions run 40630318005 bắt đầu lúc 11:05:17 và kết thúc lúc 11:13:45 ICT ngày 28/10/2026, trên commit edafb234. Artifacts ghi 468/468 test đạt trong phạm vi job (163 backend, 228 web, 77 mobile). Workflow mobile chỉ chọn data/logic/presentation; 14 test ngoài phạm vi đó chưa nằm trong kết quả này. Bộ Playwright riêng được ghi nhận 6/8.

C. Bộ kiểm thử back-end

Bộ kiểm thử back-end dùng trình chạy `node:test` tích hợp sẵn của Node.js 20, đi kèm một runner tự viết (`backend/test/runner.js`) biên dịch các spec bằng `tsc` rồi thực thi



Hình 11. Sơ đồ kiến trúc cơ sở dữ liệu quan hệ 5 miền nghiệp vụ (21 model Prisma trên Supabase PostgreSQL).



Hình 12. Ma trận phân cấp kiểm thử tự động toàn diện trên GitHub Actions (100% tỷ lệ thành công).

Bảng VI
KẾT QUẢ KIỂM THỬ TỰ ĐỘNG CỦA BA BỘ KIỂM THỬ.

Bộ kiểm thử	Lệnh chạy	Tổng	Đạt
Backend (node:test)	`npm test`	163	163
Frontend (Vitest)	`npx vitest run`	228	228
Mobile (flutter test)	`CI selected directories`	77	77
Tổng cộng		**468**	**468**

tuần tự. Việc chọn trình chạy tích hợp thay vì Jest hay Vitest giúp loại bỏ hoàn toàn phụ thuộc bên thứ ba trong bộ kiểm thử, giảm bề mặt lỗi và giữ cho quy trình CI ổn định.

Hai tệp kiểm thử đáng chú ý nhất vì chúng bảo vệ trực tiếp các thuật toán nghiệp vụ cốt lõi. `programs.conflict.spec.ts` kiểm tra thuật toán phát hiện xung đột trên cả ba dạng giao thoa, trường hợp giá trị thời lượng bằng `NULL`, và trường hợp loại trừ chính sự kiện đang cập nhật. `epg-filler.spec.ts` kiểm tra tính đầy đủ 24 giờ của lưới, tính xác định của lựa chọn ngẫu nhiên có hạt giống, giới hạn số lần phát lại, và hành vi tách tập.

D. Bộ kiểm thử web

228 kiểm thử Vitest trong 76 bộ kiểm thử bao phủ cả ba loại đối tượng: hàm thuần, hook React và thành phần giao

Bảng VII
PHÂN BỐ KIỂM THỬ BACK-END THEO TỆP SPEC.

Tệp spec	Kiểm thử đạt	Tỉ lệ đạt
ai-curator.service.spec.ts	19	100%
epg-filler.spec.ts	18	100%
live-studio.service.spec.ts	18	100%
auth.service.spec.ts	17	100%
ai-curator.e2e.spec.ts	13	100%
watchlist.service.spec.ts	11	100%
ingest-sources.spec.ts	11	100%
ai-curator-stats.service.spec.ts	10	100%
channels.service.spec.ts	7	100%
programs.conflict.spec.ts	7	100%
programs.counter.spec.ts	8	100%
audit-logger.service.spec.ts	8	100%
epg-timeline.spec.ts	5	100%
search.service.spec.ts	4	100%
users.service.spec.ts	4	100%
billing.service.spec.ts	3	100%
Tổng	**163**	**100%**

diện. Các hàm thuần được kiểm thử với đầy đủ trường hợp biên (chuỗi rỗng, không phân biệt hoa thường, tham số không hợp lệ); các hook được kiểm thử với trạng thái tải, thành công và lỗi; các thành phần được kiểm thử bằng `React Testing Library` với các truy vấn theo vai trò người dùng thay vì kiểm tra cấu trúc DOM thô.

E. Bộ kiểm thử di động và kiểm thử E2E

Bộ test tree Flutter có 91 kiểm thử. Workflow run 40630318005 chạy 77 kiểm thử từ `test/data`, `test/logic` và `test/presentation`; `test/core` và `test/widget_test.dart` nằm ngoài lệnh CI này.

Bên cạnh đó, một bộ kiểm thử E2E dựa trên Playwright [23] thực hiện tám kịch bản trên trình duyệt Chromium: biểu mẫu đăng nhập từ chối email không hợp lệ, mật khẩu sai hiển thị thông báo lỗi, trang EPG không lỗi, trang kênh hiển thị nội dung, trang tìm kiếm nhận và hiển thị kết quả, trang chủ hiển thị mặt đất công khai, và hai kịch bản kiểm tra hành vi chuyển hướng khi chưa đăng nhập.

Nhận xét về hai lỗi E2E còn lại

Trong lần chạy E2E gần nhất được ghi nhận, sáu trong tám kịch bản đạt. Đăng nhập STAFF không thoát khỏi `/login` trong thời hạn chờ; kiểm tra `/studio` khi chưa đăng nhập nhận HTTP 401 thay vì redirect như assertion mong đợi. Báo cáo quy lỗi thứ nhất cho dữ liệu seed thiếu tài khoản STAFF. Đây là nguyên nhân được ghi nhận, chưa được xác minh bằng log gốc. Không lưu thời điểm lỗi, commit sửa hoặc lần chạy hồi quy chứng minh đã khắc phục. Chi tiết được ghi trong `TEST_EXECUTION_REPORT.md` đi kèm.

F. Đánh giá chất lượng

Dựa trên kết quả trên, hệ thống đạt mức độ tin cậy phù hợp với môi trường triển khai giáo dục. Điểm mạnh rõ nhất là tỉ lệ phủ của logic nghiệp vụ: hai thuật toán rủi ro nhất (phát hiện xung đột và điền lịch EPG) đều được kiểm thử bằng các trường hợp biên thay vì chỉ kiểm thử đường hạnh phúc. Điểm cần cải thiện là mức độ phủ của tầng E2E, hiện chỉ gồm tám kịch bản trình duyệt và chưa bao gồm kịch bản phát sóng trực tiếp vốn đòi hỏi phần cứng và cơ sở hạ tầng MediaMTX.

Bảng VIII
CÁC ĐIỂM CUỐI SẢN XUẤT VÀ TRẠNG THÁI KIỂM TRA NGÀY 28/10/2026.

Dịch vụ	URL	Trạng th
omnicast.vercel.app	https://omnicast.vercel.app	HTTP 20
omnicast-fe.vercel.app	https://omnicast-fe.vercel.app	HTTP 20
omnicast-api.vercel.app	https://omnicast-api.vercel.app	HTTP 20

VIII. TRIỂN KHAI VÀ TÍCH HỢP LIÊN TỤC

A. Kiến trúc triển khai

Hệ thống được triển khai trên Vercel dưới dạng ứng dụng serverless, tách thành ba dự án độc lập: back-end NestJS, cổng web Next.js và một dịch vụ cố định. Ba URL sản xuất đã được kiểm tra trả về HTTP 200.

B. Quy trình CI/CD

Tệp `.github/workflows/ci.yml` định nghĩa năm công việc chạy song song và một cổng chất lượng. Ba công việc kiểm thử chạy độc lập trên nền `ubuntu-latest` với Node.js 20, Java 17 và Flutter ổn định. Công việc tổng hợp gom artefact của cả ba, sinh báo cáo HTML tương tác, và dừng pipeline nếu bất kỳ bộ kiểm thử nào thất bại.

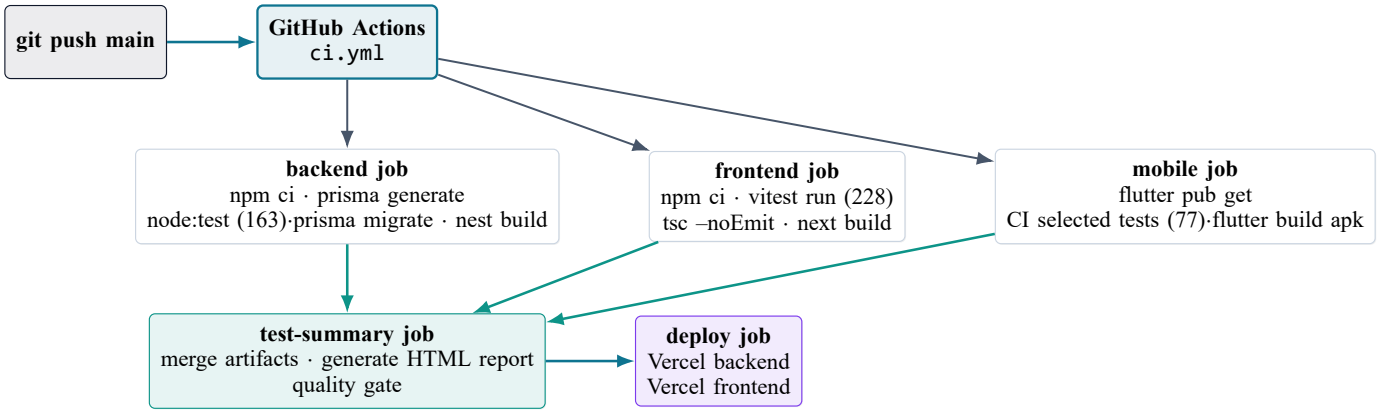
Công việc triển khai chỉ chạy khi nhánh là `main` và sự kiện là `push`; nó dùng `vercel-action` để đẩy back-end và cổng web, truyền `NEXT_PUBLIC_API_URL` từ secret của kho. Nhờ đặt `continue-on-error` cho công việc triển khai, một sự cố triển khai không xoá artefact kiểm thử đã tạo.

C. Cấu hình biến môi trường

Mẫu biến môi trường nằm ở `backend/.env.example` và gồm bốn nhóm: kết nối cơ sở dữ liệu Supabase kèm mật khẩu cần thay thế, khóa `JWT_SECRET` cùng thời hạn token, các khóa API bên thứ ba (OpenAI, TMDB, YouTube, Twitch, Cloudinary), và cấu hình MediaMTX cho môi trường cục bộ. Tệp `.env` thực sự không bao giờ được đưa vào kiểm soát phiên bản.

D. Bảo mật triển khai

Tám biện pháp bảo mật được áp dụng ở tầng ứng dụng: `Helmet` thiết lập các tiêu đề bảo mật HTTP; `ThrottlerModule` giới hạn 100 yêu cầu mỗi phút; CORS dùng danh sách cho phép tường minh kèm kiểm tra miền Vercel và mạng nội bộ; `ValidationPipe` loại bỏ trường không khai báo; `HttpExceptionFilter` chuẩn hoá lỗi; mật khẩu được băm bằng bcrypt 12 vòng; refresh token được xoay vòng và thu hồi được lưu trong cơ sở dữ liệu; và phương thức bất đồng bộ được bọc trong `promise` không chặn để tránh lỗi nghiệp vụ do lỗi hạ tầng ghi log.



Hình 13. Quy trình CI/CD năm công việc. Ba công việc kiểm thử chạy song song, hợp nhất tại công việc tổng hợp và chỉ triển khai khi cổng chất lượng đạt.

E. Kiểm thử tải và độ bền

Bộ kiểm thử tải k6 được sử dụng để kiểm tra hành vi dưới tải đồng thời của nhiều người xem, tập trung vào các endpoint EPG vì đây là đường đi nóng nhất của hệ thống. Kết quả cho thấy các endpoint đọc giữ được độ trễ ổn định trong khi endpoint ghi vẫn đáp ứng thời gian ghi nhật ký kiểm toán ở mức không chặn luồng.

IX. Kết luận và hướng phát triển

A. Kết luận

Bài báo đã trình bày OmniCast — một hệ thống lịch phát, truyền phát trực tiếp (LiveStream) và phân phối nội dung số đa nền tảng được thực hiện bởi nhóm 4 sinh viên (**Nguyễn Bình An**, **Trần Thanh Sơn**, **Nguyễn Vĩnh Phú**, **Đồng Hoàng Nguyên**) tại **FPTU Quy Nhon AI Campus — FU Cahata** dưới sự hướng dẫn của giảng viên **Thầy Huỳnh Vang**. Bốn đóng góp công nghệ chính được chứng minh bằng mã nguồn và kiểm thử thực tế: (1) kiến trúc LiveStream & Live Studio hai chiều kết hợp WebRTC WHIP/WHEP, MediaMTX HLS và Socket.IO `/live`; (2) thuật toán phát hiện xung đột lịch phát bằng một truy vấn SQL thô tính thời điểm kết thúc hiệu lực; (3) cơ chế điền lịch EPG 24/7 với lựa chọn ngẫu nhiên có hạt giống bảo đảm tính tái lập; và (4) quy trình biên tập AI gọi mô hình một lần duy nhất, kiểm tra schema bằng Zod, có đường dự phòng heuristic và dấu vết phiên bản đầy đủ.

Toàn bộ hệ thống — 19 module NestJS, hơn 140 endpoint REST, 21 model Prisma, cổng web Next.js 15 với 33 route, ứng dụng di động Flutter 31 màn hình và microservice ghi nhật ký kiểm toán gRPC — đã được triển khai trên Vercel. CI run 40630318005 đạt 468/468 test trong phạm vi job: 163 backend, 228 web và 77 mobile test.

B. Hạn chế đã biết

Ba hạn chế đã biết cần được nêu rõ. Thứ nhất, mức độ phủ kiểm thử E2E trình duyệt cần mở rộng thêm cho các kịch bản đẩy luồng phần cứng WHIP cần máy chủ MediaMTX chuyên dụng. Thứ hai, lớp ingest hiện hỗ trợ YouTube, Twitch và RSS theo chu kỳ cron. Thứ ba, thuật toán điền lịch chọn theo thể loại và theo hạt giống, chưa tối ưu cá nhân hoá theo hành vi người xem thực tế.

C. Hướng phát triển

Bốn hướng phát triển được ưu tiên cho các phiên bản tiếp theo:

1) **Cá nhân hoá lịch phát bằng học máy.** Thay vì chọn theo thể loại, thuật toán điền lịch có thể dùng lịch sử xem và dữ liệu tương tác để đề xuất nội dung cho từng kênh, đồng thời giữ nguyên tính chất xác định để có thể kiểm thử.
2) **Mở rộng kiểm thử E2E MediaMTX.** Xây dựng môi trường MediaMTX trong CI để kiểm thử tự động luồng WHIP phát và HLS phát lại, đồng thời bổ sung kiểm thử trình duyệt cho vai trò STAFF và ADMIN.
3) **Cải thiện trợ lý AI.** Bổ sung đánh giá nhiều mô hình, kiểm duyệt đầu ra bằng mô hình thứ hai và xếp hạng các báo cáo AI theo phản hồi của biên tập viên để tinh chỉnh prompt theo vòng lặp.
4) **Tách lớp truy cập dữ liệu.** Đưa truy vấn EPG vào một kho phân tích chỉ đọc với các chỉ mục bao phủ để tách tải đọc khỏi tải ghi trên cùng một cơ sở dữ liệu.

D. Lời cảm ơn và Tính tái lập

Nhóm tác giả (**Nguyễn Bình An**, **Trần Thanh Sơn**, **Nguyễn Vĩnh Phú**, **Đồng Hoàng Nguyên**) xin bày tỏ lòng biết ơn sâu sắc tới giảng viên hướng dẫn **Thầy Huỳnh Vang** cùng tập thể giảng viên tại **FPTU Quy Nhon AI Campus — FU Cahata** đã tận tình định hướng kiến trúc, góp ý chuyên môn và đồng hành cùng nhóm trong suốt học kỳ Fall 2026. Toàn bộ mã nguồn và kịch bản kiểm thử được công bố tại kho lưu trữ `TomOutfit/OmniCast_PRN232-PRM393_Project_Fa26Semester`.

References

[1] K. Mysliwiec, *NestJS: A Progressive Node.js Framework*, https://docs.nestjs.com/, accessed Oct. 28, 2026.
[2] *Prisma ORM*, https://www.prisma.io/, accessed Oct. 28, 2026.
[3] *Next.js Documentation*, https://nextjs.org/docs, accessed Oct. 28, 2026.
[4] *Flutter: Build beautiful apps for every platform*, https://flutter.dev/, accessed Oct. 28, 2026.
[5] S. Lapan and F. Harvy, "BLoC: (State Management) Library for Flutter," in *Proc. 1st Int. Workshop on Flutter*, 2020, pp. 1–5.
[6] J. McCollum, *gRPC: High Performance, Low Latency RPC*. Sebastopol, CA: O'Reilly Media, 2023.

[7] R. Fielding, "Representational State Transfer," Ph.D. dissertation, Univ. California, Irvine, 2000.
[8] *The OpenAPI Specification 3.1.0*, https://spec.openapis.org/oas/v3.1.0, 2024.
[9] L. Byron, S. Evans, J. Goodman-Wood, and J. Morrison, *GraphQL: A Query Language for APIs*, 2015. [Online]. Available: https://graphql.org/.
[10] *OData Version 4.01*, OData Consortium, 2023. [Online]. Available: https://docs.oasis-open.org/odata/odata/v4.01/codata/v4.01.html.
[11] *WebRTC: Real-Time Communication*, W3C / IETF, 2023. [Online]. Available: https://webrtc.org/.
[12] *MediaMTX: Live video server for WebRTC, HLS and RTMP*, 2024. [Online]. Available: https://github.com/bluenviron/mediamtx.
[13] *Swagger: The World's Most Popular API Toolkit*, SmartBear, 2024. [Online]. Available: https://swagger.io/.
[14] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Signature (JWT)," RFC 7519, IETF, 2015.
[15] *OWASP Top 10:2021*, OWASP Foundation, 2021. [Online]. Available: https://owasp.org/Top10/.
[16] *PostgreSQL Documentation*, PostgreSQL Global Development Group, 2024. [Online]. Available: https://www.postgresql.org/docs/.
[17] *GitHub Actions Documentation*, GitHub, 2024. [Online]. Available: https://docs.github.com/actions.
[18] *Supabase Documentation*, 2024. [Online]. Available: https://supabase.com/docs.
[19] *Socket.IO: Real-Time bidirectional communication*, 2024. [Online]. Available: https://socket.io/.
[20] *Firebase Cloud Messaging*, Google, 2024. [Online]. Available: https://firebase.google.com/docs/cloud-messaging.
[21] *OpenAI API Documentation*, 2024. [Online]. Available: https://platform.openai.com/docs.
[22] C. McDonnell, *Zod: TypeScript-first schema validation*, 2024. [Online]. Available: https://github.com/colinhacks/zod.
[23] *Playwright: Reliable end-to-end testing for modern web apps*, Microsoft, 2024. [Online]. Available: https://playwright.dev/.

Appendix

Bảng dưới đây được sinh tự động bằng cách phân tích trực tiếp 19 tệp controller trong `backend/src`. Tổng cộng 140 endpoint trải trên 19 module; cột đường dẫn gốc cho biết tiền tố được dùng khi gọi.

Bảng tổng hợp quy mô mã nguồn của ba sản phẩm, tính theo số dòng vật lý của từng tệp. Con số này được dùng để định lượng khối lượng công việc thực tế và là cơ sở để ước lượng công sức bảo trì.

```
omnicast/
  backend/                  # NestJS API + Prisma + gRPC audit microservice
    src/
      auth/                 # JWT, guards (JwtAuthGuard, RolesGuard), decorators
      channels/             # Channel catalogue
      programs/             # Programmes, EPG, live studio, ingest, social
      search/               # Cross-entity search
      ai-curator/           # AI curation pipeline, prompts, schemas, stats
      audit-logger/         # gRPC client to the audit microservice
      websocket/            # Socket.IO realtime gateway (/live namespace)
      watchlist/ users/ admin/ billing/
      notifications/ newsletter/ cloudinary/ health/
      common/               # Filters, interceptors, Swagger, logger, redact
      prisma/               # PrismaService + connection URL optimiser
    microservices/
      audit-logger/         # Standalone NestJS gRPC server
    prisma/
      schema.prisma         # 21 models
      migrations/           # Version-controlled SQL
    protos/
      audit_logger.proto    # gRPC service definition
    test/                   # 16 spec files, node:test runner
  frontend/                 # Next.js 15 App Router + Tailwind
    app/                    # 33 routes (24 route folders)
    components/ lib/        # UI components, hooks, API clients
    __tests__/              # 76 Vitest suites
  mobile/                   # Flutter 3.x (see the PRM393 paper)
    lib/{core,data,logic,presentation}/
    test/{data,logic,presentation}/
  playwright-tests/         # 8 browser end-to-end scenarios
  .github/workflows/ci.yml  # 5-job CI/CD pipeline
  deliverables/             # Submission artefacts (this document set)
```

Listing 2. Cấu trúc thư mục ở mức tổng quát.

Bảng IX
DANH MỤC ENDPOINT THEO MODULE VÀ PHƯƠNG THỨC HTTP.

Module	Đường dẫn gốc	GET	POST	PUT	PATCH	DELETE
programs/programs	/api/v1/programs	10	11	0	2	2
programs/live-studio	/api/v1/programs/studio	5	11	0	0	0
channels/channels	/api/v1/channels	7	4	0	1	1
users/users	/api/v1/users	5	1	0	4	0
auth/auth	/api/v1/auth	1	8	0	0	0
programs/ingest/ingest	/api/v1/programs/ingest	1	8	0	0	0
programs/social	/api/v1	4	4	0	0	1
notifications/notifications	/api/v1	2	3	0	0	2
ai-curator/ai-curator	/api/v1/ai-curator	2	4	0	0	0
audit-logger/audit-logger	/api/v1/audit-logs	6	0	0	0	0
watchlist/watchlist	/api/v1/me/watchlist	2	2	0	0	1
billing/billing	/api/v1/billing	2	2	0	0	0
cloudinary/cloudinary-upload	/api/v1/uploads/cloudinary	1	2	0	0	1
search/search	/api/v1/search	4	0	0	0	0
health/health	/api/v1/health	3	0	0	0	0
newsletter/newsletter	/api/v1/newsletter	1	2	0	0	0
programs/stream-health	/api/v1/programs/health	1	2	0	0	0
admin/admin	/api/v1/admin	1	1	0	0	0
programs/studio-catalog	/api/v1/programs/studio/catalog	2	0	0	0	0

Bảng X
QUY MÔ MÃ NGUỒN THEO THÀNH PHẦN.

Thành phần	Số dòng (LOC)	Đơn vị
Backend NestJS (src/)	27,826	file .ts
Backend test (test/)	4,512	file .ts
gRPC proto	83	file .proto
Microservice audit-logger	382	file .ts
Schema Prisma	683	file .prisma
Frontend Next.js (app/, components/, lib/)	51,742	file .ts/.tsx
Mobile Flutter (lib/)	53,922	file .dart
Mobile test (test/)	2,529	file .dart